MINI-PROJET DE DATA MINING

ANNÉE ACADÉMIQUE 2026/2027 · PARCOURS A : CLASSIFICATION SUPERVISÉE

Prédiction des recommandations d’achats de vêtements pour femmes

Classification supervisée des avis du jeu de données
Women’s E-Commerce Clothing Reviews

Rapport technique

COURS	Data Mining (Fouille de Données)
JEU DE DONNÉES	Women’s E-Commerce Clothing Reviews (Kaggle) : 23 486 avis bruts, 10 attributs
APPROCHE	Machine Learning classique ; aucun modèle de Deep Learning, aucun LLM
MODÈLE RETENU	Linear SVC : accuracy 0,9378 · F1-score 0,9622 · ROC-AUC 0,9775
APPLICATION	Streamlit (src/app.py), six onglets
DÉPÔT GITHUB	github.com/RSD018/ Prediction-des-recommandations--dachats--de-vetements--pour--femmes
ÉQUIPE	4 étudiants, identifiés dans les supports publics par leur code étudiant (compte GitHub du dépôt : RSD018)

Résumé

Ce rapport présente la conception, la réalisation et l'évaluation d'un pipeline complet de Data Mining qui prédit si une cliente **recommande** ou non un article de mode à partir de son avis (titre et texte) et de quelques métadonnées : âge, note, division, département et catégorie du produit. Le jeu de données, *Women's E-Commerce Clothing Reviews*, regroupe 23 486 avis bruts décrits par 10 attributs. La cible binaire `Recommended IND` est déséquilibrée : 82,2 % des avis sont des recommandations.

La démarche suit les étapes classiques de la fouille de données. L'exploration et le nettoyage traitent les valeurs manquantes, les doublons et les valeurs atypiques (règle de Tukey) ; deux tests statistiques, le test de Mann-Whitney U et le test du χ^2 d'indépendance, étudient le lien entre la cible et, respectivement, la note et le département. Le prétraitement s'appuie sur un `ColumnTransformer` (imputation, standardisation, encodage one-hot, TF-IDF) ajusté sur le seul jeu d'entraînement, ce qui évite toute fuite de données et produit une matrice creuse d'environ 13 000 variables. Six approches sont ensuite comparées : un baseline Zero-R, K-NN, un arbre de décision, une forêt aléatoire, un Naïve Bayes multinomial et un SVM linéaire, les cinq modèles réglables étant optimisés par `GridSearchCV` avec validation croisée stratifiée. Le meilleur modèle est déployé dans une application Streamlit à six onglets.

Le **Linear SVC** obtient les meilleurs résultats (accuracy 0,9378 ; F1-score 0,9622 ; ROC-AUC 0,9775), très au-dessus du baseline Zero-R (accuracy 0,8223). Une reconstitution algébrique des matrices de confusion à partir des métriques publiées montre cependant que la classe minoritaire « non recommandé » reste la plus difficile à détecter (rappel estimé à environ 0,82 pour le modèle retenu). Ce constat oriente les perspectives : rééquilibrage des classes, réglage du seuil de décision, représentations textuelles enrichies. Aucun modèle de Deep Learning ni aucun LLM n'est utilisé comme modèle prédictif.

Mots-clés : Data Mining ; classification supervisée ; classification de texte ; TF-IDF ; SVM linéaire ; GridSearchCV ; déséquilibre de classes ; fuite de données ; Streamlit.

Table des matières

Résumé 2

1 Introduction 9

1.1 Contexte et motivation 9

1.2 Problématique 9

1.3 Objectifs du projet 9

1.4 Périmètre et contraintes du cahier des charges 10

1.5 Organisation du rapport 11

2 Cadre théorique et notions préliminaires 12

2.1 Démarche de fouille de données : CRISP-DM 12

2.2 Apprentissage supervisé et classification binaire 12

2.3 Validation croisée et recherche d'hyperparamètres 13

2.4 Données déséquilibrées 14

2.5 Métriques d'évaluation 15

2.6 Représentation du texte : sac de mots et TF-IDF 17

2.7 Préparation des variables numériques et catégorielles 18

2.8 Fuite de données (data leakage) 19

2.9 Tests statistiques et analyses exploratoires 20

3 Présentation du jeu de données 23

3.1 Source, licence et contexte 23

3.2 Dimensions et conformité à la volumétrie exigée 23

3.3 Dictionnaire des variables 23

3.4 Qualité des données : valeurs manquantes et doublons 24

3.5 Variable cible et déséquilibre des classes 25

3.6 Considérations éthiques et limites des données 25

4 Organisation du projet et environnement technique 26

4.1 Structure du dépôt 26

4.2 Gestion de version et travail collaboratif 27

4.3 Environnement technique 28

4.4 Reproductibilité 28

5 Exploration, tests statistiques et nettoyage 30

5.1 Démarche du notebook 01 . . . 30
5.2 Typage des variables et conséquences . . . 30
5.3 Qualité des données : valeurs manquantes et doublons . . . 31
5.4 Statistiques descriptives et distributions . . . 31
5.5 Valeurs atypiques (règle de Tukey) . . . 32
5.6 Analyse de la cible et corrélations . . . 32
5.7 Tests statistiques et justification de leur choix . . . 32
5.8 Procédure de nettoyage . . . 34
5.9 Bilan du chapitre . . . 34

6 Prétraitement et encodage des variables . . . 35
6.1 Séparation X / y et suppression de Clothing ID . . . 35
6.2 Découpage entraînement / test stratifié . . . 35
6.3 Le ColumnTransformer . . . 36
6.4 Absence de fuite de données . . . 37
6.5 Matrices creuses et coût mémoire . . . 38
6.6 Artefacts sérialisés . . . 38
6.7 Bilan du chapitre . . . 38

7 Modélisation et optimisation des hyperparamètres . . . 39
7.1 Protocole expérimental . . . 39
7.2 Les algorithmes comparés . . . 39
7.3 Recherche des hyperparamètres avec GridSearchCV . . . 43
7.4 Sélection du modèle final . . . 44
7.5 Synthèse comparative des algorithmes . . . 44

8 Résultats et évaluation . . . 45
8.1 Choix des métriques . . . 45
8.2 Résultats comparatifs sur le jeu de test . . . 45
8.3 Lecture des résultats . . . 46
8.4 Analyse par classe : reconstitution des matrices de confusion . . . 47
8.5 Courbes ROC et AUC . . . 50
8.6 Positionnement par rapport à des références externes et rôle de la note . . . 50
8.7 Bilan du chapitre . . . 51

9 Application Streamlit . . . 52
9.1 Architecture et principes de fonctionnement . . . 52
9.2 Les six onglets . . . 52

9.3 Chaîne de prédiction en direct . . . 53
9.4 Exécution et déploiement . . . 53
9.5 Évolutions envisagées . . . 54

10 Discussion, limites et perspectives . . . 55
10.1 Retour sur les questions du projet . . . 55
10.2 Limites du travail . . . 55
10.3 Perspectives . . . 56
10.4 Responsabilité et usage . . . 56

11 Conclusion . . . 57

Références bibliographiques . . . 58

A Procédure de reproduction pas à pas . . . 59
A.1 Installation de l'environnement . . . 59
A.2 Régénération des données et des modèles . . . 59

B Dérivation des métriques reconstituées . . . 60
B.1 Hypothèses et relations de départ . . . 60
B.2 Démonstration . . . 60
B.3 Application numérique au Linear SVC . . . 60
B.4 Incertitude due à l'arrondi . . . 61

C Glossaire . . . 62

Liste des figures

Figure 2.1 — Sous-apprentissage et surapprentissage en fonction de la complexité du modèle (schéma illustratif). 13

Figure 2.2 — Validation croisée stratifiée à K = 5 plis à l'intérieur du jeu d'entraînement ; le jeu de test reste à l'écart. 14

Figure 2.3 — Matrice de confusion d'un classifieur binaire (classe positive : « recommandé »). 15

Figure 2.4 — Courbes ROC schématiques : plus la courbe épouse le coin supérieur gauche, plus l'AUC est élevée (valeurs illustratives). 16

Figure 2.5 — Matrice TF-IDF normalisée du corpus fictif (les points indiquent des valeurs nulles). 18

Figure 2.6 — Loi du χ^2 à 5 degrés de liberté : la zone colorée est la région de rejet de H0 au seuil de 5 %. 21

Figure 2.7 — Diagramme en boîte et limites de Tukey sur dix âges fictifs : la valeur 99 est atypique. 22

Figure 3.1 — Nombre de valeurs manquantes par colonne dans le jeu brut (source : métadonnées OpenML du jeu de données). 24

Figure 3.2 — Répartition de la variable cible ; la ligne pointillée indique l'accuracy du baseline Zero-R sur le jeu de test. 25

Figure 4.1 — Vue d'ensemble du pipeline : des données brutes à l'application Streamlit. 26

Figure 6.1 — Découpage stratifié 80 % / 20 % : les proportions de classes sont identiques dans les trois ensembles. 35

Figure 6.2 — Structure du ColumnTransformer : trois branches parallèles assemblées en une matrice creuse. . . 36

Figure 7.1 — Principe du K-NN avec K = 5 : le nouvel avis (étoile) est classé selon la majorité de ses cinq plus proches voisins (données synthétiques). 40

Figure 7.2 — Indice de Gini et entropie d'un nœud binaire en fonction de la proportion p de la classe 1 ; le point correspond au nœud racine du projet. 41

Figure 7.3 — Principe du SVM linéaire : hyperplan séparateur (trait plein), marges (pointillés) et vecteurs de support entourés (données synthétiques). 43

Figure 8.1 — Comparaison des modèles sur le jeu de test : accuracy, F1-score et ROC-AUC (axe tronqué à partir de 0,80). 46

Figure 8.2 — Rappel de la classe 1 et rappel de la classe 0 (spécificité) de chaque modèle, valeurs reconstituées (± 0,01). 48

Figure 8.3 — Matrice de confusion reconstituée du Linear SVC, en pourcentage du jeu de test (incertitude d'environ ± 0,2 point). 49

Figure 9.1 — Chaîne de traitement de l'application : du formulaire à la prédiction affichée. 52

Liste des tableaux

Tableau 1.1 — Correspondance entre les exigences du cahier des charges et les réalisations du projet 10

Tableau 2.1 — Les six phases de CRISP-DM et leur mise en œuvre dans le projet 12

Tableau 2.2 — Stratégies face au déséquilibre de classes 15

Tableau 2.3 — Synthèse des métriques et valeurs obtenues par le baseline Zero-R 17

Tableau 2.4 — Fréquences et idf du corpus fictif de trois avis (n = 3) 18

Tableau 2.5 — Sensibilité des algorithmes à l’échelle et aux matrices creuses 19

Tableau 2.6 — Sources courantes de fuite de données et bonnes pratiques 19

Tableau 2.7 — Choix d’un test selon la nature des variables 20

Tableau 2.8 — Exemple fictif de calcul des rangs pour le test de Mann-Whitney 21

Tableau 2.9 — Exemple fictif de tableau de contingence 3 × 2 (effectifs observés et attendus) 21

Tableau 3.1 — Conformité du jeu de données aux exigences de volumétrie 23

Tableau 3.2 — Dictionnaire des variables du jeu de données brut 23

Tableau 3.3 — Valeurs manquantes du jeu brut (23 486 avis), d’après les métadonnées publiques OpenML . . . 24

Tableau 4.1 — Rôle de chaque élément du dépôt 27

Tableau 4.2 — Lots de travail du projet et livrables associés 28

Tableau 4.3 — Principales bibliothèques et versions figées dans requirements.txt 28

Tableau 4.4 — Chaîne de traitement : entrées et sorties de chaque étape 29

Tableau 5.1 — Type statistique des variables et conséquences pour l’analyse 30

Tableau 5.2 — Stratégies de traitement des valeurs manquantes 31

Tableau 5.3 — Comparaison des deux tests statistiques du projet 33

Tableau 6.1 — Traitement appliqué à chaque famille de variables 36

Tableau 6.2 — Ce que le préprocesseur apprend sur l’entraînement et la façon dont il traite le test 37

Tableau 6.3 — Fichiers sérialisés du dossier models/ 38

Tableau 7.1 — Les six approches comparées 39

Tableau 7.2 — Hyperparamètres principaux de chaque algorithme 44

Tableau 7.3 — Synthèse des six approches 44

Tableau 8.1 — Performances des modèles sur le jeu de test (source : README du dépôt) 45

Tableau 8.2 — Apport de chaque modèle par rapport au baseline Zero-R (accuracy 0,8223 ; F1 0,9025) 46

Tableau 8.3 — Écarts avec le Linear SVC exprimés en erreurs-types (jeu de test supposé de 4 700 avis) 47

Tableau 8.4 — Métriques par classe reconstituées à partir de l'accuracy, du F1 et de p = 0,8223 (incertitude ± 0,01) .

Tableau 8.5 — Classement des cinq modèles appris selon la métrique retenue (valeurs reconstituées pour le macro-F1 et l'accuracy équilibrée) 49

Tableau 8.6 — F1-scores de références externes et du projet (jeux de test différents : comparaison indicative) . . 50

Tableau 9.1 — Les six onglets de l'application 53

Tableau 10.1 — Réponses aux cinq questions du projet 55

Tableau 10.2 — Limites identifiées et mesures correctives 55

Tableau 10.3 — Perspectives classées par priorité 56

Tableau A.1 — Fichiers produits par chaque notebook et contrôles de cohérence 59

Tableau B.1 — Reconstitution détaillée de la matrice de confusion du Linear SVC 60

Tableau C.1 — Glossaire des termes techniques 62

1 Introduction

1.1 Contexte et motivation

Dans le commerce en ligne, les avis clients constituent l'une des sources de données les plus riches pour comprendre la satisfaction, repérer les défauts d'un produit ou orienter les ventes. Parmi les informations laissées par une cliente, l'indicateur « je recommande ce produit » est particulièrement précieux : il résume en un seul bit un jugement global, plus directement exploitable qu'une note moyenne ou qu'un commentaire libre.

Cet indicateur n'est toutefois pas toujours disponible : case facultative dans un formulaire, avis recueillis sur d'autres canaux, textes issus des réseaux sociaux. Il peut aussi sembler contredire le texte, par exemple lorsqu'une note élevée accompagne un commentaire réservé. Savoir prédire automatiquement la recommandation à partir de l'avis et de quelques métadonnées permet de compléter des données lacunaires, de détecter des incohérences et de prioriser le traitement des avis défavorables.

Le présent projet s'inscrit dans le mini-projet de Data Mining de l'année 2026/2027. Il porte sur le jeu de données public *Women's E-Commerce Clothing Reviews* et suit le **parcours A** du cahier des charges : la classification supervisée, réalisée exclusivement avec des algorithmes de Machine Learning classiques.

1.2 Problématique

La question centrale est la suivante : **peut-on prédire de façon fiable si une cliente recommande un article de mode à partir du titre et du texte de son avis, de sa note et de quelques caractéristiques du produit, en n'utilisant que des algorithmes de Machine Learning classiques ?** Elle se décline en cinq questions auxquelles les chapitres suivants répondent tour à tour :

1. Quelles variables sont statistiquement liées à la recommandation ? (chapitre chapitre 5)
2. Comment combiner dans un même modèle des variables numériques, catégorielles et textuelles sans introduire de fuite de données ? (chapitre chapitre 6)
3. Parmi les algorithmes du cours, lequel offre le meilleur compromis entre performance et coût de calcul sur des données textuelles de grande dimension ? (chapitre chapitre 7)
4. Comment évaluer honnêtement des modèles lorsque 82,2 % des avis appartiennent à la même classe ? (chapitre chapitre 8)
5. Comment rendre le modèle utilisable par une personne non spécialiste ? (chapitre chapitre 9)

1.3 Objectifs du projet

1.3.1 Objectifs scientifiques

- explorer et documenter le jeu de données : types des variables, qualité, distributions, corrélations ;
- réaliser au moins deux tests statistiques scientifiquement appropriés et justifier explicitement leur choix ;
- construire un pipeline de prétraitement reproductible et exempt de fuite de données ;
- comparer six approches (Zero-R, K-NN, Decision Tree, Random Forest, Multinomial Naïve Bayes, Linear SVC) avec une optimisation systématique des hyperparamètres ;
- analyser les résultats avec des métriques adaptées à des classes déséquilibrées.

1.3.2 Objectifs applicatifs

- sérialiser le meilleur modèle et son préprocesseur pour les réutiliser sans ré-entraînement ;
- fournir une application Streamlit, exécutable en local, qui prédit la recommandation en temps réel et présente les résultats de l'étude ;
- documenter le dépôt GitHub (structure, installation, exécution) afin que l'ensemble soit reproductible.

1.4 Périmètre et contraintes du cahier des charges

Le cahier des charges du mini-projet fixe des règles précises : taille minimale du jeu de données, choix d'un parcours technique, algorithmes obligatoires, interdiction du Deep Learning et des LLM comme modèles de Machine Learning, optimisation systématique des hyperparamètres, tests statistiques, application interactive, dépôt GitHub public structuré et anonymisation des supports. Le Tableau 1.1 met en regard chaque exigence et la réalisation correspondante dans ce projet.

***Tableau 1.1** — Correspondance entre les exigences du cahier des charges et les réalisations du projet*

Exigence du cahier des charges	Réalisation dans le projet
Jeu de données d'au moins 10 000 enregistrements bruts	23 486 avis bruts (`data/raw/reviews.csv`)
Au moins 10 attributs	10 attributs, plus une colonne d'index technique (section section 3.3)
Parcours technique A : classification supervisée	Classification binaire de la cible `Recommended IND`
Algorithmes du cours : baseline Zero-R, K-NN, arbres et forêts, Naïve Bayes, SVM	Zero-R, K-NN, Decision Tree, Random Forest, Multinomial Naïve Bayes, Linear SVC
Deep Learning et LLM interdits comme modèles ML	Aucun modèle de Deep Learning ni LLM ; aucune bibliothèque correspondante dans `requirements.txt`
Optimisation systématique (GridSearchCV ou RandomizedSearchCV, K ≥ 5)	GridSearchCV avec validation croisée stratifiée pour les modèles réglables (Zero-R exempté)
Au moins 2 tests statistiques justifiés	Test de Mann-Whitney U (Rating × Recommended IND) et test du χ^2 d'indépendance (Department Name × Recommended IND)
Prétraitement : valeurs manquantes, encodage, normalisation	Imputation, standardisation Z-score, encodage one-hot, TF-IDF, sans fuite de données
Comparaison multicritère des modèles	Accuracy, F1-score, ROC-AUC ; analyse par classe (section section 8.4)
Application interactive exécutable en local	Application Streamlit (`src/app.py`), six onglets
Dépôt GitHub public et arborescence standardisée	`data/raw`, `data/processed`, `notebooks/`, `src/`, `docs/`, `requirements.txt` (+ `models/` pour les artefacts)
Anonymat sur les supports publics	Aucun nom ni prénom : seuls les codes étudiants sont utilisés

1.5 Organisation du rapport

Le rapport suit la logique d’un projet de fouille de données. Le chapitre chapitre 2 rassemble les notions théoriques nécessaires (processus CRISP-DM, validation croisée, métriques, TF-IDF, tests statistiques). Le chapitre chapitre 3 présente le jeu de données et le chapitre chapitre 4 l’organisation du dépôt et de l’environnement. Les chapitres chapitre 5 et chapitre 6 détaillent l’exploration, le nettoyage et le prétraitement ; le chapitre chapitre 7 décrit les algorithmes et leur optimisation ; le chapitre chapitre 8 analyse les résultats. Le chapitre chapitre 9 présente l’application, et le chapitre chapitre 10 discute des limites et des perspectives avant la conclusion (chapitre chapitre 11). Les annexes regroupent la procédure de reproduction, la dérivation des métriques reconstituées et un glossaire.

> **Convention de lecture**
>
> Les noms de fichiers, de fonctions et de colonnes apparaissent en `police à chasse fixe`. Les nombres décimaux suivent la notation française (virgule). Les chiffres de performance proviennent du README du dépôt ; les valeurs marquées « reconstituées » sont déduites algébriquement de ces chiffres (section section 8.4), et les exemples pédagogiques utilisent des données fictives signalées comme telles.

2 Cadre théorique et notions préliminaires

Ce chapitre rassemble les notions nécessaires pour comprendre les choix techniques du projet. Chaque notion est introduite ici de façon autonome, puis appliquée aux données dans les chapitres suivants ; les renvois sont indiqués dans le texte. Les exemples numériques de ce chapitre utilisent des données fictives, recalculées avec scikit-learn et SciPy afin que chaque chiffre soit exact.

2.1 Démarche de fouille de données : CRISP-DM

Un projet de Data Mining n'est pas une simple succession d'appels à des bibliothèques : c'est un processus itératif. Deux cadres de référence le décrivent : le processus KDD (*Knowledge Discovery in Databases*) et la méthode CRISP-DM (*Cross-Industry Standard Process for Data Mining*, Chapman et al., 2000). Nous retenons CRISP-DM, dont les six phases se retrouvent dans l'organisation du dépôt (Tableau 2.1).

***Tableau 2.1** — Les six phases de CRISP-DM et leur mise en œuvre dans le projet*

Phase	Question posée	Réalisation dans le projet
1. Compréhension du métier	Quel problème résoudre, avec quel critère de succès ?	Prédire la recommandation d'un article ; critères : F1-score et ROC-AUC (chapitres chapitre 1 et chapitre 8)
2. Compréhension des données	De quelles données dispose-t-on ? Sont-elles fiables ?	Jeu Kaggle de 23 486 avis ; exploration et diagnostic de qualité (chapitres chapitre 3 et chapitre 5)
3. Préparation des données	Comment obtenir une table exploitable par les algorithmes ?	Nettoyage, imputation, standardisation, encodage one-hot, TF-IDF (chapitres chapitre 5 et chapitre 6)
4. Modélisation	Quels algorithmes, quels réglages ?	Six approches comparées, GridSearchCV (chapitre chapitre 7)
5. Évaluation	Le modèle répond-il à l'objectif ? Est-il fiable ?	Jeu de test isolé, métriques adaptées au déséquilibre (chapitre chapitre 8)
6. Déploiement	Comment l'utiliser concrètement ?	Application Streamlit, artefacts sérialisés avec joblib (chapitre chapitre 9)

Le processus est par nature itératif : l'évaluation peut conduire à revenir sur la préparation des données. C'est d'ailleurs l'une des perspectives de ce travail (chapitre chapitre 10).

2.2 Apprentissage supervisé et classification binaire

2.2.1 Formalisation

En classification supervisée binaire, on dispose d'un jeu d'apprentissage $D = \{(x_i, y_i)\}$, pour $i = 1, \ldots, n$. Le vecteur $x_i \in \mathbb{R}^d$ contient les variables explicatives d'un individu (ici, un avis) et $y_i \in \{0, 1\}$ est sa classe (ici, 1 si la cliente recommande l'article, 0 sinon). On cherche une fonction f qui prédise correctement la classe d'individus nouveaux, jamais vus pendant l'apprentissage. Un algorithme choisit f dans une famille de fonctions (arbres, hyperplans, etc.) en minimisant le **risque empirique** :

$$R_{\text{emp}}(f) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}[f(x_i) \neq y_i] \tag{2.1}$$

où $\mathbf{1}[\cdot]$ vaut 1 lorsque la condition est vraie et 0 sinon. Minimiser le seul risque empirique ne suffit pas : ce qui compte est l'erreur commise sur des données nouvelles (le *risque réel*), que l'on estime sur un jeu de test tenu à l'écart.

2.2.2 Sous-apprentissage et surapprentissage

Un modèle trop simple (**sous-apprentissage**) ne capture pas la structure des données : son erreur est élevée à la fois sur l'entraînement et sur la validation. Un modèle trop flexible (**surapprentissage**) mémorise le bruit : l'erreur d'entraînement devient très faible alors que l'erreur sur de nouvelles données remonte (Figure 2.1). Les hyperparamètres (profondeur d'un arbre, nombre de voisins K, constante C d'un SVM, etc.) servent précisément à régler cette complexité.

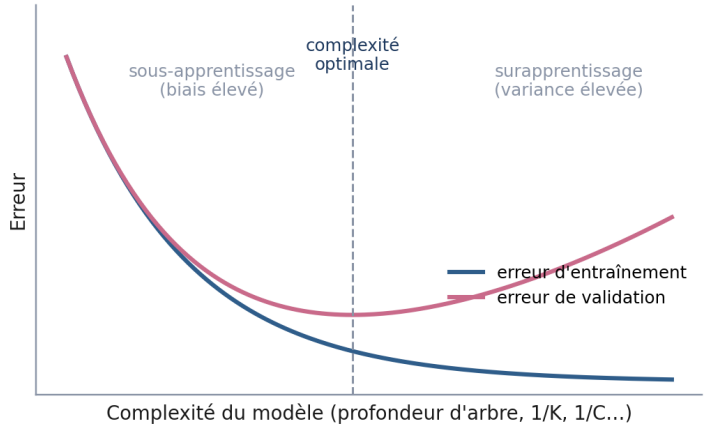


Figure 2.1 *— Sous-apprentissage et surapprentissage en fonction de la complexité du modèle (schéma illustratif).*

Trois sous-ensembles jouent des rôles distincts : le jeu d'**apprentissage** ajuste les paramètres du modèle ; la **validation**, réalisée par validation croisée à l'intérieur du jeu d'entraînement, sert à choisir les hyperparamètres ; le jeu de **test** estime la performance finale et n'est utilisé qu'une seule fois.

2.3 Validation croisée et recherche d'hyperparamètres

Les *paramètres* d'un modèle (coefficients d'un hyperplan, seuils d'un arbre) sont appris à partir des données. Les *hyperparamètres* (K, profondeur maximale, C, α, etc.) sont fixés avant l'apprentissage et pilotent la complexité. Pour les choisir sans jamais toucher au jeu de test, on utilise la **validation croisée en K plis** : le jeu d'entraînement est découpé en K sous-ensembles de tailles voisines (les plis) ; chaque pli sert à tour de rôle de validation, tandis que les K − 1 autres servent à l'apprentissage ; le score final est la moyenne des K scores obtenus :

$$\mathrm{CV}(\theta) = \frac{1}{K} \sum_{k=1}^{K} \mathrm{score}\left(f_{\theta}^{(-k)}, D_k\right) \tag{2.2}$$

où $f_{\theta}^{(-k)}$ est le modèle d'hyperparamètres θ entraîné sans le pli k et D_k le pli de validation. Dans la variante **stratifiée**, chaque pli conserve la proportion des classes du jeu complet. Cette précaution est indispensable lorsque la classe minoritaire ne représente que 17,8 % des observations : sans elle, certains plis pourraient contenir très peu d'exemples de cette classe (Figure 2.2).

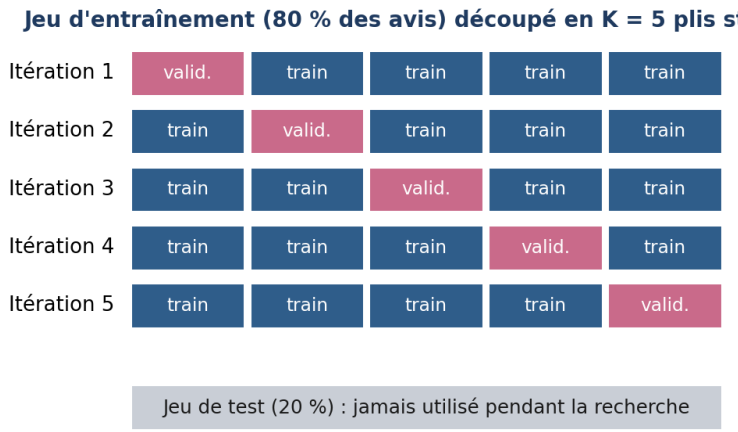


***Figure 2.2** — Validation croisée stratifiée à K = 5 plis à l'intérieur du jeu d'entraînement ; le jeu de test reste à l'écart.*

`GridSearchCV` automatise cette démarche. Pour chaque combinaison d'hyperparamètres d'une grille, il réalise K apprentissages, calcule le score moyen de validation, retient la meilleure combinaison, puis ré-entraîne le modèle sur l'ensemble du jeu d'entraînement (étape de *refit*). Le coût est proportionnel à la taille de la grille multipliée par K : une grille de 4 × 3 = 12 combinaisons avec K = 5 plis représente 12 × 5 + 1 = 61 apprentissages. Le critère de sélection est fixé par le paramètre `scoring` (accuracy par défaut ; F1-score ou ROC-AUC sont des alternatives courantes). `RandomizedSearchCV` tire au hasard un nombre fixé de combinaisons et convient mieux aux grilles très grandes. Le cahier des charges impose K ≥ 5 et l'une de ces deux méthodes pour tous les modèles réglables.

> **Règle d'or**
>
> Le jeu de test n'intervient ni dans le choix des hyperparamètres, ni dans le prétraitement. Il est utilisé une seule fois, à la fin, pour la comparaison finale des modèles.

2.4 Données déséquilibrées

Un jeu de données est **déséquilibré** lorsque les classes ne sont pas représentées à parts égales. Ici, la classe 1 (« recommandé ») représente 82,2 % des avis et la classe 0 (« non recommandé ») 17,8 %, soit un rapport d'environ 4,6 contre 1. Cela a trois conséquences.

1. **L'accuracy est trompeuse.** Un classifieur qui prédit toujours la classe majoritaire (Zero-R) atteint déjà une accuracy égale à la proportion de cette classe, soit 82,23 % sur le jeu de test du projet.
2. **Les algorithmes tendent à favoriser la classe majoritaire.** Minimiser l'erreur globale peut se faire au détriment du rappel de la classe minoritaire.
3. **Les découpages doivent être stratifiés** pour que chaque sous-ensemble conserve les mêmes proportions.

Tableau 2.2 — Stratégies face au déséquilibre de classes

Stratégie	Principe	Statut dans le projet
Découpages stratifiés	Conserver les proportions de classes dans chaque sous-ensemble	Utilisée (split train/test et validation croisée)
Métriques adaptées	F1-score, ROC-AUC et rappel par classe plutôt que l'accuracy seule	Utilisée (chapitre chapitre 8)
Pondération des classes	Pénaliser davantage les erreurs sur la classe minoritaire (`class_weight`)	Perspective (chapitre chapitre 10)
Ré-échantillonnage	Sous-échantillonner la classe majoritaire ou sur-échantillonner la minoritaire (SMOTE, Chawla et al., 2002)	Perspective
Réglage du seuil	Déplacer le seuil de décision pour privilégier le rappel de la classe 0	Perspective

Notons que SMOTE fabrique des exemples synthétiques par interpolation entre voisins ; appliqué à des matrices TF-IDF creuses de plus de dix mille colonnes, il produit des vecteurs qui ne correspondent à aucun texte réel, ce qui limite son intérêt dans ce projet.

2.5 Métriques d'évaluation

La **matrice de confusion** croise les classes réelles et les classes prédites (Figure 2.3). Par convention, la classe positive est ici la classe 1 (« recommandé ») : VP désigne les vrais positifs, VN les vrais négatifs, FP les faux positifs (avis non recommandé prédit recommandé) et FN les faux négatifs (avis recommandé prédit non recommandé).

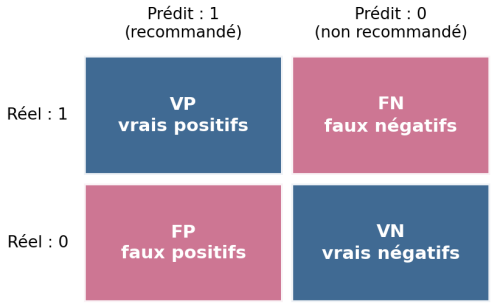


Figure 2.3 — Matrice de confusion d'un classifieur binaire (classe positive : « recommandé »).

Les métriques usuelles se déduisent de ces quatre effectifs.

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN} \tag{2.3}$$

$$\text{Précision} = \frac{VP}{VP + FP} \qquad \text{Rappel} = \frac{VP}{VP + FN} \tag{2.4}$$

$$\text{Spécificité} = \frac{VN}{VN + FP} \tag{2.5}$$

$$F_1 = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}} = \frac{2\,VP}{2\,VP + FP + FN} \tag{2.6}$$

La **précision** répond à la question « quand le modèle annonce "recommandé", a-t-il raison ? » ; le **rappel** (ou sensibilité) à « quelle part des avis réellement recommandés est retrouvée ? » ; la **spécificité** est le

rappel de la classe 0. Le **F1-score** est la moyenne harmonique de la précision et du rappel : plus sévère que la moyenne arithmétique, elle reste basse dès que l'une des deux composantes est faible.

> **Un F1 flatteur lorsque la classe positive est majoritaire**
>
> Le baseline Zero-R ne détecte aucun avis non recommandé, mais obtient pourtant un F1 de 0,9025 : sa précision vaut p (la proportion de la classe 1) et son rappel vaut 1, donc F1 = 2p/(1 + p) = 2 × 0,8223 / 1,8223 ≈ 0,9025. Le F1 de la classe 1 est donc utile mais insuffisant ; il est complété par l'AUC et par le rappel de chaque classe.

Deux métriques agrégées corrigent en partie ce biais : l'**accuracy équilibrée**, moyenne des rappels des deux classes, et le **macro-F1**, moyenne des F1 calculés pour chaque classe.

$$\text{Accuracy}_{\text{équilibrée}} = \frac{\text{Rappel} + \text{Spécificité}}{2} \tag{2.7}$$

Enfin, un classifieur produit en général un *score* s(x) (probabilité, distance à l'hyperplan, etc.) ; la prédiction « classe 1 » correspond à s(x) > seuil. En faisant varier le seuil, on trace le taux de vrais positifs (rappel) en fonction du taux de faux positifs (1 − spécificité) : c'est la **courbe ROC** (Fawcett, 2006, Figure 2.4). L'**aire sous la courbe** (AUC) mesure la capacité du score à classer correctement les avis :

$$\text{AUC} = P(s(x^+) > s(x^-)) = \frac{U}{n_+ n_-} \tag{2.8}$$

C'est la probabilité qu'un avis positif tiré au hasard reçoive un score supérieur à celui d'un avis négatif tiré au hasard (0,5 pour le hasard, 1 pour un classement parfait). Elle ne dépend pas du seuil et est peu sensible au déséquilibre. Elle est égale à la statistique U de Mann-Whitney normalisée par $n_+ n_-$ (section section 2.9.2).

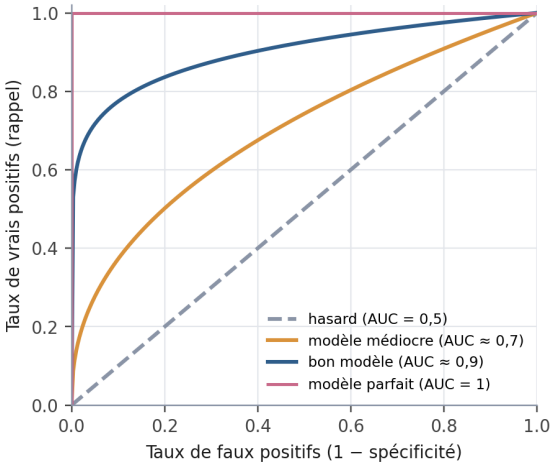


***Figure 2.4** — Courbes ROC schématiques : plus la courbe épouse le coin supérieur gauche, plus l'AUC est élevée (valeurs illustratives).*

Tableau 2.3 — Synthèse des métriques et valeurs obtenues par le baseline Zero-R

Métrique	Question à laquelle elle répond	Zero-R
Accuracy	Quelle part des avis est bien classée ?	0,8223
Précision (classe 1)	Quand le modèle annonce « recommandé », a-t-il raison ?	0,8223
Rappel (classe 1)	Quelle part des avis recommandés est retrouvée ?	1,0000
Spécificité (rappel classe 0)	Quelle part des avis non recommandés est détectée ?	0,0000
F1-score (classe 1)	Quel compromis entre précision et rappel pour la classe 1 ?	0,9025
Accuracy équilibrée	Quelle est la moyenne des rappels des deux classes ?	0,5000
ROC-AUC	Le score classe-t-il bien les avis, quel que soit le seuil ?	0,5 (hasard)

2.6 Représentation du texte : sac de mots et TF-IDF

Les algorithmes du cours opèrent sur des vecteurs numériques : il faut donc transformer le texte. La représentation par **sac de mots** (*bag-of-words*) ignore l'ordre des mots et compte, pour chaque document, les occurrences de chaque terme d'un vocabulaire. Elle comporte quatre étapes : la *tokenisation* (découpage en mots), la *normalisation* (passage en minuscules, éventuellement suppression des mots vides), la construction du *vocabulaire* V sur le jeu d'entraînement, puis celle de la *matrice document-terme*.

Compter les mots favorise les termes très fréquents mais peu informatifs (« the », « and », « it »). La pondération **TF-IDF** (Salton et Buckley, 1988) corrige cet effet : elle multiplie la fréquence du terme dans le document (tf) par un facteur qui décroît avec le nombre de documents contenant ce terme (idf). Dans la version « lissée » de scikit-learn, utilisée par défaut par `TfidfVectorizer`, on a :

$$\mathrm{idf}(t) = \ln \frac{1+n}{1+\mathrm{df}(t)} + 1 \qquad \mathrm{tfidf}(t,d) = \mathrm{tf}(t,d) \times \mathrm{idf}(t) \tag{2.9}$$

où n est le nombre de documents du corpus d'apprentissage et df(t) le nombre de documents qui contiennent t. Chaque vecteur est ensuite normalisé en norme L2, ce qui rend comparables des avis courts et des avis longs :

$$\mathbf{v}_d \leftarrow \frac{\mathbf{v}_d}{\|\mathbf{v}_d\|_2} \tag{2.10}$$

Exemple fictif. Considérons un corpus de trois avis : (1) « love this dress love the fit », (2) « poor fit and poor quality » et (3) « love the quality ». Avec n = 3, le terme « love » apparaît dans 2 documents : idf = ln(4/3) + 1 = 1,2877. Le terme « poor », présent dans un seul document, a un idf plus élevé (ln(4/2) + 1 = 1,6931), car il est plus discriminant (Tableau 2.4).

Tableau 2.4 — Fréquences et idf du corpus fictif de trois avis (n = 3)

Terme	df	idf	tf avis 1	tf avis 2	tf avis 3
and	1	1,6931	0	1	0
dress	1	1,6931	1	0	0
fit	2	1,2877	1	1	0
love	2	1,2877	2	0	1
poor	1	1,6931	0	2	0
quality	2	1,2877	0	1	1
the	2	1,2877	1	0	1
this	1	1,6931	1	0	0

Pour l'avis 1, le poids brut de « love » est tf × idf = 2 × 1,2877 = 2,5754. La norme du vecteur brut de cet avis vaut 3,9601 ; le poids final est donc 2,5754 / 3,9601 = **0,650**, valeur que l'on retrouve dans la Figure 2.5. Dans l'avis 3, les trois termes ont le même idf : leurs poids finaux sont égaux (1/√3 ≈ 0,577).

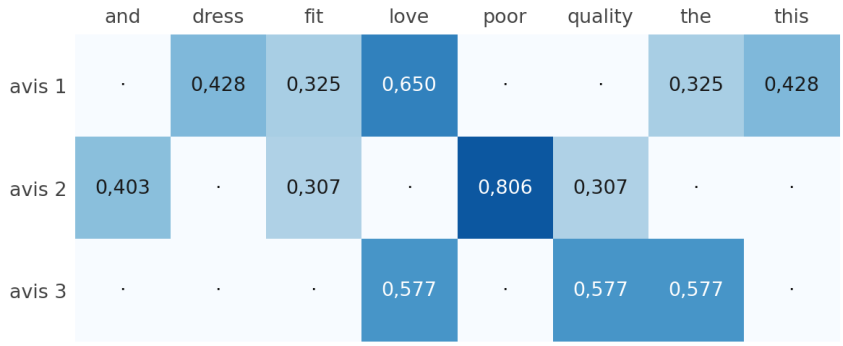


Figure 2.5 — Matrice TF-IDF normalisée du corpus fictif (les points indiquent des valeurs nulles).

- **Matrice creuse.** Un avis ne contient que quelques dizaines de termes distincts, alors que le vocabulaire en compte plusieurs milliers : la quasi-totalité des cases vaut zéro. On ne stocke que les valeurs non nulles (format CSR de `scipy.sparse`), ce qui réduit fortement la mémoire et le temps de calcul.
- **Vocabulaire appris sur l'entraînement.** Les mots présents uniquement dans le jeu de test sont ignorés ; c'est une condition pour éviter la fuite de données (section section 2.8).
- **Limites.** Le sac de mots perd l'ordre des mots, donc la négation (« not good » contre « good »), l'ironie et la polysémie. Les *n-grammes* (suites de n mots consécutifs) atténuent en partie ce défaut.

2.7 Préparation des variables numériques et catégorielles

Imputation. Une valeur manquante peut être remplacée par une statistique apprise sur l'entraînement : la *médiane* pour une variable numérique (robuste aux valeurs extrêmes, contrairement à la moyenne) ou la valeur *la plus fréquente* pour une variable catégorielle. Une alternative consiste à créer une modalité explicite « Unknown », qui conserve l'information que la valeur était absente.

Standardisation (Z-score). Elle recentre et réduit chaque variable numérique :

$$z = \frac{x - \mu}{\sigma} \tag{2.11}$$

où μ et σ sont la moyenne et l'écart-type calculés sur l'entraînement. Pour les valeurs fictives {25, 34, 41, 52, 60}, on obtient μ = 42,4 et σ = 12,47, d'où les scores z = {-1,40; -0,67; -0,11; 0,77; 1,41}. Les

algorithmes fondés sur des distances (K-NN) ou sur une marge (SVM) y sont sensibles : sans standardisation, une variable exprimée en dizaines (l'âge) écraserait une variable bornée entre 1 et 5 (la note). Les arbres et les forêts, invariants par transformation monotone, n'en ont pas besoin.

Encodage one-hot. Une variable catégorielle à m modalités est remplacée par m colonnes binaires, dont une seule vaut 1 pour chaque individu. Par exemple, `Department Name` (6 modalités) donne 6 colonnes. Contrairement à un codage entier (0, 1, 2…), cet encodage n'impose aucun ordre artificiel entre les modalités ; son coût est l'augmentation du nombre de colonnes. Une modalité absente de l'entraînement doit être gérée au moment du test (ignorée, c'est-à-dire codée par un vecteur nul).

__Tableau 2.5__ — Sensibilité des algorithmes à l'échelle et aux matrices creuses

Algorithme	Sensible à l'échelle ?	Accepte les matrices creuses ?	Remarque
K-NN	Oui (distances)	Oui	Standardisation indispensable pour les variables numériques
Linear SVC	Oui (marge)	Oui	Solveur adapté aux données creuses de grande dimension
Decision Tree	Non	Oui	Invariant aux transformations monotones
Random Forest	Non	Oui	Idem ; moyenne de nombreux arbres
Multinomial NB	Non	Oui	Exige des valeurs d'entrée positives (comptages, poids TF-IDF)

2.8 Fuite de données (data leakage)

Il y a **fuite de données** lorsque des informations issues du jeu de test (ou de la cible) influencent l'apprentissage ou le prétraitement. Les performances mesurées sont alors artificiellement optimistes et ne se reproduiront pas en production. Le Tableau 2.6 recense les situations les plus fréquentes.

__Tableau 2.6__ — Sources courantes de fuite de données et bonnes pratiques

Mauvaise pratique	Conséquence	Bonne pratique
Standardiser tout le jeu de données avant le découpage	La moyenne et l'écart-type intègrent des informations du test	Ajuster le scaler sur l'entraînement uniquement, puis transformer le test
Construire le vocabulaire et les idf sur tous les avis	Les mots rares du test influencent les poids	`fit` sur l'entraînement, `transform` sur le test
Choisir les hyperparamètres avec le jeu de test	Le test devient un jeu de validation : estimation biaisée	Validation croisée à l'intérieur de l'entraînement
Laisser des doublons entre entraînement et test	Le modèle est évalué sur des exemples déjà vus	Supprimer les doublons avant le découpage
Utiliser une variable dérivée de la cible ou connue après coup	Le modèle « triche »	Vérifier que chaque variable est disponible au moment de la prédiction

Dans ce projet, toutes les transformations sont ajustées (`fit`) uniquement sur le jeu d'entraînement ; le jeu de test est ensuite seulement transformé (`transform`). La mise en œuvre est décrite au chapitre chapitre 6.

2.9 Tests statistiques et analyses exploratoires

2.9.1 Principe des tests d'hypothèses

Un test d'hypothèses oppose une **hypothèse nulle** H_0 (absence d'effet, d'association ou de différence) à une **hypothèse alternative** H_1. On calcule une statistique de test dont la loi sous H_0 est connue, puis une **p-valeur** : la probabilité d'observer un résultat au moins aussi extrême si H_0 était vraie. Si $p < \alpha$, où le seuil α est fixé a priori (ici $\alpha = 0{,}05$), on rejette H_0. Deux erreurs sont possibles : l'erreur de type I (rejeter H_0 à tort, de probabilité α) et l'erreur de type II (ne pas rejeter H_0 à tort).

Avec plusieurs dizaines de milliers d'observations, la puissance des tests est telle que des écarts minimes deviennent « significatifs ». La p-valeur doit donc toujours être accompagnée d'une mesure de la **taille d'effet** (V de Cramér pour le χ^2, probabilité de supériorité pour Mann-Whitney).

__Tableau 2.7__ — Choix d'un test selon la nature des variables

Situation	Test retenu	Alternative écartée et raison
Comparer une variable ordinale ou non normale entre deux groupes indépendants	Mann-Whitney U	Test t de Student : suppose des données d'intervalle et approximativement normales ; la note de 1 à 5 est ordinale et discrète
Mesurer l'association entre deux variables catégorielles	χ^2 d'indépendance	Corrélation de Pearson : inadaptée aux variables nominales
Comparer trois groupes ou plus	ANOVA ou Kruskal-Wallis	Non utilisé dans ce projet
Mesurer une relation linéaire entre deux variables quantitatives	Corrélation de Pearson	Spearman si la relation est monotone ou si les variables sont ordinales

2.9.2 Test de Mann-Whitney U

Le test de Mann-Whitney (ou Wilcoxon-Mann-Whitney, Mann et Whitney, 1947) compare la position de deux distributions indépendantes sans supposer leur normalité. H_0 : les deux groupes proviennent de la même distribution ; H_1 : l'un des groupes a tendance à prendre des valeurs plus grandes. Il se fonde sur les *rangs* : on fusionne les deux échantillons, on attribue à chaque observation son rang (rang moyen en cas d'égalités), puis on somme les rangs R_1 du premier groupe :

$$U_1 = R_1 - \frac{n_1(n_1+1)}{2} \qquad U_2 = n_1 n_2 - U_1 \tag{2.12}$$

Pour de grands échantillons, U suit approximativement une loi normale de moyenne $n_1 n_2/2$; la p-valeur s'obtient alors à partir de la statistique centrée réduite, avec une correction pour les égalités. Le rapport $U_1/(n_1 n_2)$ est la **probabilité de supériorité** : la probabilité qu'une observation du groupe 1 dépasse une observation du groupe 2 (les égalités comptant pour moitié). Il vaut 0,5 en l'absence de différence, et c'est exactement l'AUC d'un classifieur qui utiliserait la variable comme score (équation équation (2.8)).

Exemple fictif. Deux groupes de six notes : le groupe A (« recommande ») et le groupe B (« ne recommande pas »). Après fusion, les valeurs triées sont 1, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5 ; les rangs moyens sont donc 1 pour la note 1, 2,5 pour la note 2, 5 pour la note 3, 8 pour la note 4 et 11 pour la note 5 (Tableau 2.8).

Tableau 2.8 — Exemple fictif de calcul des rangs pour le test de Mann-Whitney

Groupe	Notes observées	Rangs correspondants	Somme des rangs
A ($n_1 = 6$)	5, 4, 5, 3, 5, 4	11, 8, 11, 5, 11, 8	54
B ($n_2 = 6$)	2, 3, 1, 3, 4, 2	2,5, 5, 1, 5, 8, 2,5	24

On obtient $R_1 = 54$, d'où $U_1 = 54 - 6 \times 7 / 2 = 33$ et $U_2 = 36 - 33 = 3$. La probabilité de supériorité vaut 33 / 36 = 0,917 ; la p-valeur (approximation normale, test bilatéral, calculée avec SciPy) est d'environ 0,017 : au seuil de 5 %, on rejetterait H_0 dans cet exemple.

2.9.3 Test du χ² d'indépendance

Le test du χ² (Pearson, 1900) évalue l'association entre deux variables catégorielles à partir de leur tableau de contingence à r lignes et c colonnes. H_0 : les deux variables sont indépendantes. Sous H_0, l'effectif attendu de la cellule (i, j) est $E_{ij} = L_i \, C_j / N$ (total de la ligne i fois total de la colonne j, divisé par l'effectif total). La statistique mesure l'écart entre effectifs observés O_{ij} et attendus :

$$\chi^2 = \sum_{i=1}^{r} \sum_{j=1}^{c} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \qquad \text{ddl} = (r-1)(c-1) \tag{2.13}$$

Elle suit approximativement une loi du χ² à (r − 1)(c − 1) degrés de liberté ; l'approximation est valable si tous les effectifs attendus sont supérieurs ou égaux à 5 (sinon, on regroupe des modalités ou l'on recourt au test exact de Fisher). La taille d'effet est mesurée par le **V de Cramér** :

$$V = \sqrt{\frac{\chi^2}{N(\min(r,c)-1)}} \in [0,1] \tag{2.14}$$

Pour un département à 6 modalités croisé avec une cible binaire, le nombre de degrés de liberté est (6 − 1)(2 − 1) = 5 (6 si la modalité « Unknown » est conservée), et la valeur critique à α = 5 % est 11,07 (Figure 2.6).

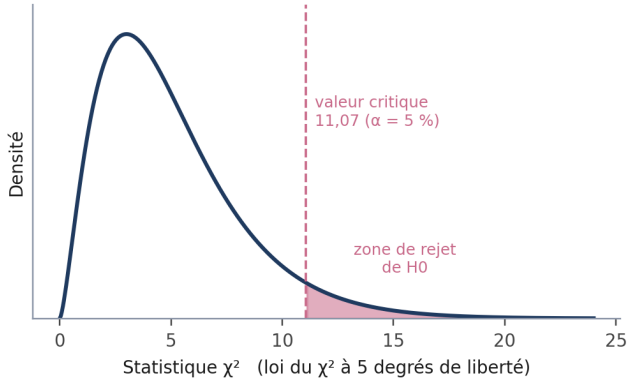


Figure 2.6 — Loi du χ² à 5 degrés de liberté : la zone colorée est la région de rejet de H0 au seuil de 5 %.

Exemple fictif. Trois départements A, B et C, avec 300 avis au total (Tableau 2.9).

Tableau 2.9 — Exemple fictif de tableau de contingence 3 × 2 (effectifs observés et attendus)

Département	Recommandé (observé)	Non recommandé (observé)	Recommandé (attendu)	Non recommandé (attendu)
A	80	20	66,67	33,33
B	70	30	66,67	33,33
C	50	50	66,67	33,33

La statistique vaut $\chi^2 = 21{,}0$ avec 2 degrés de liberté, soit une p-valeur d'environ $2{,}8 \times 10^{-5}$; le V de Cramér est 0,26 (association modérée). On rejette donc H_0 : dans cet exemple, la recommandation dépend du département.

2.9.4 Corrélation de Pearson

Le coefficient de corrélation linéaire de Pearson entre deux variables quantitatives x et y se calcule ainsi :

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\,\sqrt{\sum_i (y_i - \bar{y})^2}} \in [-1, 1] \tag{2.15}$$

Une valeur proche de ±1 indique une forte relation linéaire, une valeur proche de 0 l'absence de relation *linéaire* (une relation non linéaire peut subsister). Le coefficient est sensible aux valeurs extrêmes. Pour des variables ordinales (la note) ou monotones, on préfère le coefficient de Spearman, qui est le coefficient de Pearson des rangs. Lorsque l'une des variables est binaire (la cible), le coefficient de Pearson devient la corrélation point-bisérial. Une matrice de corrélation permet aussi de repérer les variables redondantes.

2.9.5 Règle de Tukey pour les valeurs atypiques

La règle de Tukey (1977) repose sur l'écart interquartile $\mathrm{IQR} = Q_3 - Q_1$. Une observation est jugée atypique si elle sort de l'intervalle :

$$[\,Q_1 - 1{,}5\,\mathrm{IQR}\;;\; Q_3 + 1{,}5\,\mathrm{IQR}\,] \tag{2.16}$$

Les quartiles étant des statistiques d'ordre, la règle est robuste et ne suppose aucune loi particulière. **Exemple fictif** : pour les dix âges {22, 25, 27, 30, 31, 33, 35, 38, 41, 99}, on a $Q_1 = 27{,}75$, $Q_3 = 37{,}25$, IQR = 9,5, d'où les limites 13,5 et 51,5 ; seul l'âge 99 sort de l'intervalle (Figure 2.7).

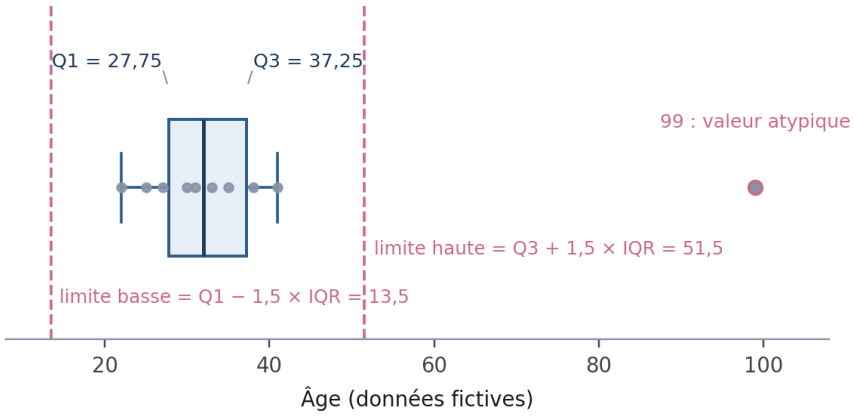


***Figure 2.7** — Diagramme en boîte et limites de Tukey sur dix âges fictifs : la valeur 99 est atypique.*

Une valeur atypique n'est pas forcément une erreur : il faut juger de sa plausibilité avant de la supprimer, de la remplacer (winsorisation) ou de la conserver.

3 Présentation du jeu de données

3.1 Source, licence et contexte

Le jeu de données *Women's E-Commerce Clothing Reviews* est publié sur la plateforme Kaggle par Nick Brooks (Brooks, 2018). Il rassemble des avis réels de clientes sur des vêtements pour femmes, déposés sur le site d'un détaillant de mode en ligne. Chaque ligne correspond à **un avis** : une cliente, un produit, une note, un titre, un texte et quelques informations de catalogue.

S'agissant de données commerciales réelles, elles ont été **anonymisées** : les références au détaillant dans le titre et le texte des avis ont été remplacées par le mot « retailer ». Le jeu est diffusé sous une licence de type domaine public (Creative Commons) et il est largement utilisé pour l'analyse de sentiments et la classification de texte, ce qui facilite la comparaison avec d'autres travaux. Les avis sont rédigés en **anglais**.

3.2 Dimensions et conformité à la volumétrie exigée

Le fichier brut `data/raw/reviews.csv` compte **23 486 lignes** et **11 colonnes**. Dix d'entre elles sont de vrais attributs ; la onzième est, dans le fichier Kaggle d'origine, une colonne d'index sans nom (`Unnamed: 0`) créée lors d'un export, sans signification pour le problème, et qui n'est pas utilisée comme variable explicative. Le jeu satisfait les seuils du cahier des charges (Tableau 3.1), et sa taille de quelques mégaoctets reste très inférieure à la limite de 100 Mo au-delà de laquelle un stockage externe serait nécessaire : le fichier est donc conservé dans le dépôt, dans `data/raw/`.

__Tableau 3.1__ — Conformité du jeu de données aux exigences de volumétrie

Critère	Exigence du cahier des charges	Jeu de données	Verdict
Enregistrements bruts	≥ 10 000	23 486	Conforme
Attributs	≥ 10	10 (hors colonne d'index)	Conforme
Taille du fichier	Stockage externe si > 100 Mo	Quelques Mo	Stockage dans le dépôt

3.3 Dictionnaire des variables

Le Tableau 3.2 décrit chaque colonne : son type statistique, sa signification, le nombre de valeurs distinctes du jeu brut (d'après les métadonnées publiques du jeu de données sur OpenML) et son traitement dans le pipeline. Le type statistique conditionne tout le reste : les statistiques descriptives pertinentes, les tests applicables, l'encodage et le choix des visualisations.

__Tableau 3.2__ — Dictionnaire des variables du jeu de données brut

Variable	Type	Description	Valeurs distinctes	Traitement dans le pipeline
Clothing ID	Identifiant	Identifiant du produit évalué	1 206	Retirée avant la modélisation (section section 6.1)
Age	Quantitative (discrète)	Âge de la cliente (de 18 à 99 ans)	77	Imputation par la médiane, standardisation
Title	Textuelle	Titre de l'avis	13 992	TF-IDF (vectoriseur dédié)
Review Text	Textuelle	Corps de l'avis	22 634	TF-IDF (vectoriseur dédié)

Variable	Type	Description	Valeurs distinctes	Traitement dans le pipeline
Rating	Ordinale (1 à 5)	Note donnée au produit	5	Imputation par la médiane, standardisation
Recommended IND	Binaire (cible)	1 : la cliente recommande ; 0 : elle ne recommande pas	2	Cible y (non transformée)
Positive Feedback Count	Quantitative (discrète)	Nombre d'autres clientes ayant jugé l'avis utile (de 0 à 122)	82	Imputation par la médiane, standardisation
Division Name	Catégorielle	Division du produit (General, General Petite, Initmates*)	3	Imputation (plus fréquent), one-hot
Department Name	Catégorielle	Département (Bottoms, Dresses, Intimate, Jackets, Tops, Trend)	6	Imputation (plus fréquent), one-hot
Class Name	Catégorielle	Sous-catégorie du produit (Dresses, Knits, Blouses, Jeans, etc.)	20	Imputation (plus fréquent), one-hot

* « Initmates » est l'orthographe d'origine de la modalité dans le jeu de données ; elle est conservée telle quelle.

3.4 Qualité des données : valeurs manquantes et doublons

Le diagnostic de qualité du jeu brut montre que seules cinq colonnes comportent des valeurs manquantes (Figure 3.1 et Tableau 3.3).

***Tableau 3.3** — Valeurs manquantes du jeu brut (23 486 avis), d'après les métadonnées publiques OpenML*

Variable	Valeurs manquantes	Part du jeu brut	Interprétation
Title	3 810	16,22 %	Titre facultatif non renseigné par la cliente
Review Text	845	3,60 %	Avis sans commentaire (note seule)
Division Name	14	0,06 %	Produit sans division
Department Name	14	0,06 %	Produit sans département
Class Name	14	0,06 %	Produit sans sous-catégorie
Les cinq autres colonnes	0	0 %	Clothing ID, Age, Rating, Recommended IND, Positive Feedback Count

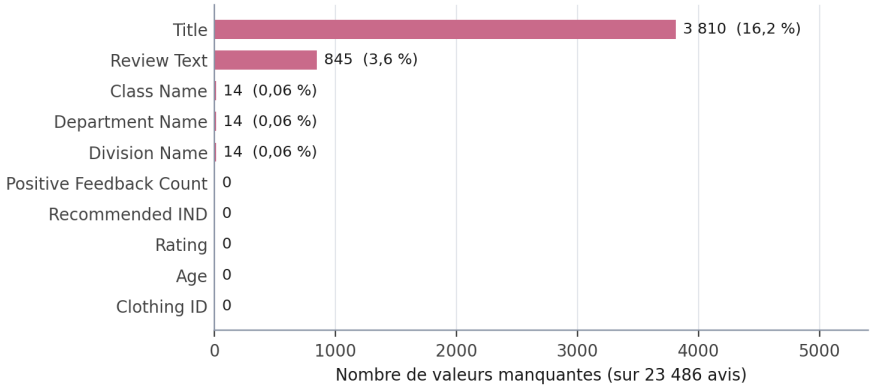


***Figure 3.1** — Nombre de valeurs manquantes par colonne dans le jeu brut (source : métadonnées OpenML du jeu de données).*

Trois enseignements s'en dégagent.

- **La cible est complète.** `Recommended IND` n'a aucune valeur manquante : aucune ligne n'est perdue faute d'étiquette, et la règle « supprimer les lignes à cible manquante » du nettoyage est ici une simple

précaution.

- **L'absence de titre ou de texte est une information, pas une erreur.** Une cliente peut laisser une note sans rédiger de commentaire. Traiter ces champs comme du texte vide (ou comme une modalité « Unknown ») conserve ces avis, alors que les supprimer retirerait près de 16 % du jeu pour le seul titre.
- **Les trois colonnes de catégorie ont chacune 14 valeurs manquantes** (0,06 %). L'égalité des effectifs suggère des produits qui n'ont pas été rattachés au catalogue, mais seule une vérification sur les données (notebook 01) permettrait de confirmer que ce sont les mêmes lignes.

Pour les **doublons**, les décomptes publiés indiquent 22 634 textes d'avis distincts pour 22 641 avis qui possèdent un texte (23 486 − 845). Les avis dupliqués sont donc très rares : sept au plus parmi les avis avec texte. Leur suppression avant le découpage train/test reste néanmoins nécessaire pour éviter qu'un même avis se retrouve à la fois dans l'entraînement et dans le test (section section 2.8).

3.5 Variable cible et déséquilibre des classes

La cible `Recommended IND` prend la valeur 1 pour 82,2 % des avis et 0 pour 17,8 %. Sur le jeu brut, cela correspond à environ 19 300 avis recommandés contre environ 4 200 non recommandés, soit un rapport d'environ 4,6 contre 1 (Figure 3.2).

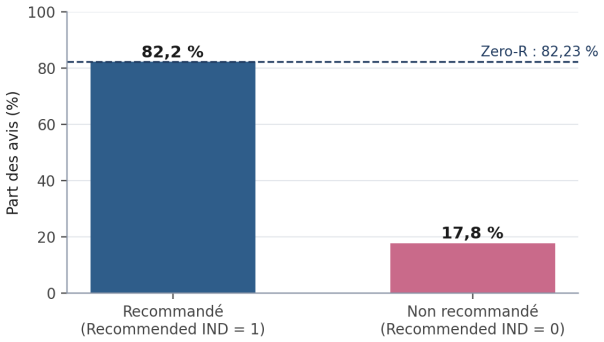


***Figure 3.2** — Répartition de la variable cible ; la ligne pointillée indique l'accuracy du baseline Zero-R sur le jeu de test.*

Ce déséquilibre conditionne la suite du projet. Un modèle qui prédirait systématiquement « recommandé » obtiendrait déjà 82,23 % d'accuracy sur le jeu de test : toute amélioration doit être mesurée par rapport à ce plancher, et non par rapport à 50 %. Il justifie le recours à un découpage stratifié (chapitre chapitre 6) et le choix du F1-score et de la ROC-AUC comme métriques principales (chapitre chapitre 8).

3.6 Considérations éthiques et limites des données

- **Anonymat.** Le jeu ne contient aucun identifiant de cliente ; la seule caractéristique personnelle est l'âge. Le nom du détaillant a été masqué dans les textes.
- **Représentativité.** Les avis proviennent d'un seul détaillant et sont rédigés en anglais. Un modèle entraîné sur ces données ne se généralise pas sans précaution à d'autres langues, d'autres enseignes ou d'autres catégories de produits.
- **Équité.** L'âge figure parmi les variables explicatives. Une étude d'équité (performances par tranche d'âge) serait nécessaire avant tout usage réel ; elle fait partie des perspectives (chapitre chapitre 10).
- **Usage.** Le projet a une vocation académique : les prédictions de l'application sont indicatives et ne doivent pas servir à prendre des décisions concernant des personnes.

4 Organisation du projet et environnement technique

Un projet de Data Mining n'est exploitable que s'il est **reproductible** : un tiers doit pouvoir retrouver les mêmes résultats à partir des mêmes données. Ce chapitre décrit l'organisation du dépôt GitHub, la façon dont l'équipe a travaillé, les bibliothèques utilisées et la procédure de reproduction.

La Figure 4.1 donne une vue d'ensemble de la chaîne de traitement, qui sert de fil conducteur à l'ensemble du rapport.

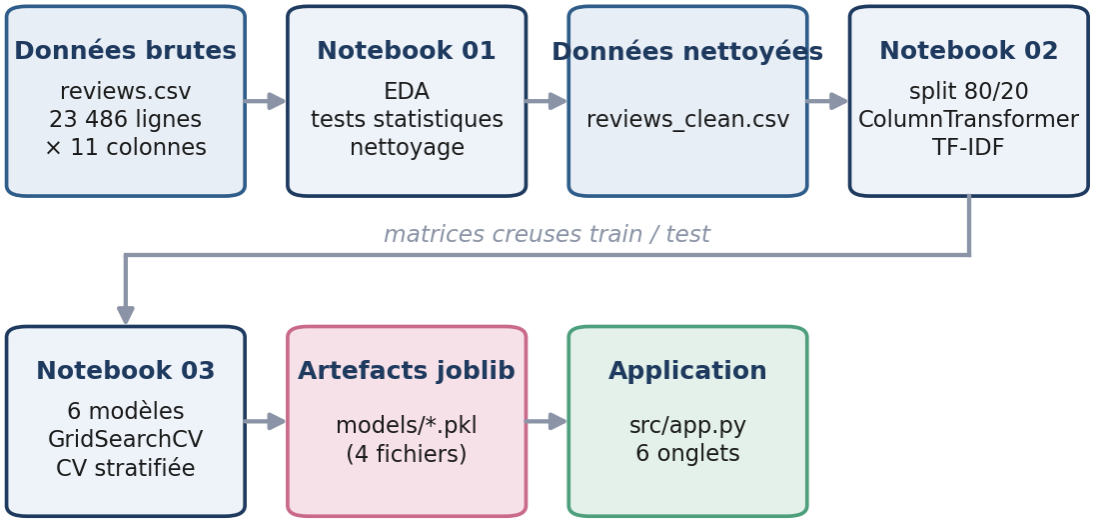


***Figure 4.1** — Vue d'ensemble du pipeline : des données brutes à l'application Streamlit.*

4.1 Structure du dépôt

Le dépôt est public et organisé de manière à séparer nettement les données, les notebooks, les modèles sérialisés, le code de l'application et la documentation. Cette organisation respecte l'arborescence recommandée par le cahier des charges (`data/`, `notebooks/`, `src/`, `docs/`, `requirements.txt`) et y ajoute un dossier `models/` pour les artefacts entraînés.

```
Prediction-des-recommandations--dachats--de-vetements--pour--femmes/
├── data/
│   ├── raw/
│   │   └── reviews.csv                  # dataset brut (23 486 lignes, 11 colonnes)
│   └── processed/
│       └── reviews_clean.csv            # dataset nettoyé (notebook 01)
├── docs/
│   └── Rapport_MiniProject.pdf          # rapport écrit du mini-projet
├── models/
│   ├── best_model.pkl                   # modèle final (Linear SVC)
│   ├── preprocessor.pkl                 # ColumnTransformer (imputation, scaling, OHE, TF-IDF)
│   ├── review_tfidf.pkl                 # vectoriseur TF-IDF (Review Text)
│   └── title_tfidf.pkl                  # vectoriseur TF-IDF (Title)
├── notebooks/
│   ├── 01_eda_and_cleaning.ipynb        # exploration, nettoyage, tests statistiques
│   ├── 02_preprocessing_encoding.ipynb  # split train/test, encodage, TF-IDF
│   └── 03_classification_models.ipynb   # entraînement, comparaison, sauvegarde du modèle
├── src/
│   ├── app.py                           # application Streamlit (démo interactive)
│   └── config.toml                      # thème Streamlit (couleurs, mode clair)
├── .gitignore
├── README.md
└── requirements.txt
```

Le Tableau 4.1 précise le rôle de chaque élément et le chapitre qui le détaille.

***Tableau 4.1** — Rôle de chaque élément du dépôt*

Élément	Rôle	Détaillé au
`data/raw/reviews.csv`	Jeu de données brut, jamais modifié	chapitre chapitre 3
`data/processed/reviews_clean.csv`	Jeu nettoyé, produit par le notebook 01	chapitre chapitre 5
`notebooks/01_eda_and_cleaning.ipynb`	Exploration, tests statistiques, nettoyage	chapitre chapitre 5
`notebooks/02_preprocessing_encoding.ipynb`	Découpage train/test, ColumnTransformer, TF-IDF	chapitre chapitre 6
`notebooks/03_classification_models.ipynb`	Six approches, GridSearchCV, sauvegarde du champion	chapitre chapitre 7
`models/*.pkl`	Modèle final, préprocesseur et vectoriseurs TF-IDF sérialisés (joblib)	chapitres chapitre 6 et chapitre 9
`src/app.py`, `src/config.toml`	Application Streamlit et thème visuel	chapitre chapitre 9
`docs/`	Rapport du mini-projet (ce document)	—
`requirements.txt`, `.gitignore`	Dépendances figées ; fichiers exclus du suivi git (environnement virtuel)	section section 4.3

4.2 Gestion de version et travail collaboratif

Le projet est entièrement hébergé sur un dépôt GitHub public, conformément au cahier des charges. L'historique de la branche principale `main` comptait 13 commits à la date de rédaction de ce rapport. Le travail s'est déroulé en plusieurs temps : mise en place de la structure et des dépendances sur des branches de fonctionnalité fusionnées par *pull request*, puis développement des notebooks et de l'application, enfin rédaction du README et du présent rapport.

Le cahier des charges évalue la contribution de chaque membre à travers l'historique Git (commits, lignes ajoutées ou supprimées, pull requests fusionnées). Dans les supports publics, les étudiants ne sont identifiés que par leur code étudiant, conformément à la règle d'anonymisation ; le compte GitHub propriétaire du dépôt est `RSD018`.

Tableau 4.2 — Lots de travail du projet et livrables associés

Lot de travail	Contenu	Livrables
L1. Mise en place	Structure du dépôt, dépendances, exclusions git	Arborescence, `requirements.txt`, `.gitignore`
L2. Exploration et nettoyage	Qualité, distributions, outliers, corrélations, tests statistiques, nettoyage	Notebook 01, `reviews_clean.csv`
L3. Prétraitement	Découpage stratifié, ColumnTransformer, TF-IDF	Notebook 02, `preprocessor.pkl`, vectoriseurs
L4. Modélisation	Six approches, GridSearchCV, évaluation, sélection du champion	Notebook 03, `best_model.pkl`
L5. Application	Interface Streamlit, thème	`src/app.py`, `src/config.toml`
L6. Documentation	README, rapport technique	`README.md`, `docs/Rapport_MiniProject.pdf`

4.3 Environnement technique

Le projet est écrit en Python et n'utilise que des bibliothèques de Machine Learning classique : aucune bibliothèque de Deep Learning (TensorFlow, PyTorch, Keras) ne figure dans les dépendances. Le fichier `requirements.txt` compte 126 lignes et fige les versions exactes de toutes les dépendances, y compris transitives. Ce choix est essentiel ici : les fichiers `.pkl` produits par `joblib` ne sont garantis compatibles qu'avec la version de scikit-learn qui les a créés. Le Tableau 4.3 présente les bibliothèques principales.

Tableau 4.3 — Principales bibliothèques et versions figées dans requirements.txt

Bibliothèque	Version	Rôle dans le projet
scikit-learn	1.9.1	Pipeline, ColumnTransformer, TF-IDF, algorithmes, GridSearchCV, métriques
pandas	3.0.5	Chargement, exploration et nettoyage des données tabulaires
NumPy	2.5.3	Calcul numérique sur tableaux
SciPy	1.18.1	Tests statistiques (Mann-Whitney, χ^2) et matrices creuses
matplotlib / seaborn	3.11.2 / 0.13.2	Visualisations des notebooks
joblib	1.6.0	Sérialisation du modèle, du préprocesseur et des vectoriseurs
Streamlit	1.64.0	Application web interactive
JupyterLab	4.6.3	Exécution des notebooks

Le dossier de l'environnement virtuel (`venv/` ou `.venv/`) est exclu du suivi git par le fichier `.gitignore` : seul `requirements.txt` est versionné, ce qui permet à chacun de recréer l'environnement.

4.4 Reproductibilité

La procédure complète est décrite à l'annexe A. En résumé, après avoir cloné le dépôt, créé un environnement virtuel et installé les dépendances, il suffit de placer `reviews.csv` dans `data/raw/` puis d'exécuter les trois notebooks dans l'ordre. Chaque étape consomme la sortie de la précédente (Tableau 4.4), ce qui rend la chaîne de traitement explicite.

__Tableau 4.4__ — Chaîne de traitement : entrées et sorties de chaque étape

Étape	Notebook	Entrée	Sortie
1. Nettoyage	01_eda_and_cleaning	data/raw/reviews.csv	data/processed/reviews_clean.csv
2. Prétraitement	02_preprocessing_encoding	reviews_clean.csv	matrices creuses train/test, models/preprocessor.pkl
3. Modélisation	03_classification_models	matrices train/test	models/best_model.pkl
4. Démonstration	src/app.py	models/*.pkl	interface Streamlit

Deux mécanismes garantissent des résultats identiques d'une exécution à l'autre : une graine aléatoire fixée (`random_state=42`) pour les modèles à composante aléatoire, c'est-à-dire Decision Tree, Random Forest et Linear SVC ; et un découpage train/test **stratifié** (`stratify=y`).

5 Exploration, tests statistiques et nettoyage

Ce chapitre décrit le travail du notebook `01_eda_and_cleaning.ipynb`. L'analyse exploratoire (EDA, *Exploratory Data Analysis*) répond à une question simple : **que contiennent réellement les données, et sont-elles prêtes pour la modélisation ?** Elle précède toujours le choix des modèles, car elle détermine les tests applicables, le traitement des valeurs manquantes et les risques à surveiller.

5.1 Démarche du notebook 01

Le notebook enchaîne huit étapes, de la compréhension à la sauvegarde du jeu nettoyé :

1. exploration structurelle et typage des variables (quantitatives, catégorielles, textuelles, cible) ;
2. analyse des valeurs manquantes et des doublons ;
3. statistiques descriptives et distributions (histogrammes, courbes de densité KDE) ;
4. détection des valeurs atypiques par la règle de Tukey (IQR) ;
5. analyse de la variable cible et de son déséquilibre ;
6. matrice de corrélation de Pearson ;
7. deux tests statistiques : Mann-Whitney U et χ^2 d'indépendance ;
8. nettoyage et sauvegarde dans `data/processed/reviews_clean.csv`.

5.2 Typage des variables et conséquences

Identifier correctement le **type statistique** de chaque variable est la première décision de l'analyse, car elle conditionne les résumés numériques, les graphiques, les tests et l'encodage (Tableau 5.1).

__Tableau 5.1__ — Type statistique des variables et conséquences pour l'analyse

Type	Variables	Résumés pertinents	Visualisation	Encodage dans le pipeline
Quantitative discrète	Age, Positive Feedback Count	Moyenne, médiane, écart-type, quartiles	Histogramme, KDE, boîte	Imputation (médiane) + standardisation
Ordinale	Rating (1 à 5)	Médiane, mode, effectifs par niveau	Diagramme en barres	Traitée comme numérique (standardisée)
Nominale	Division, Department, Class	Effectifs, mode	Diagramme en barres	Imputation (plus fréquent) + one-hot
Textuelle	Title, Review Text	Longueur, termes fréquents	Nuage ou barres de termes	TF-IDF
Binaire (cible)	Recommended IND	Proportion de chaque classe	Diagramme en barres	Aucun (cible y)
Identifiant	Clothing ID	Nombre de valeurs distinctes	—	Supprimé

Traiter la note comme une variable numérique est un choix usuel, mais il suppose que les écarts entre niveaux successifs sont comparables (passer de 1 à 2 vaut passer de 4 à 5). C'est pourquoi les tests portant sur cette variable doivent être des tests de rangs (section section 5.7.1).

5.3 Qualité des données : valeurs manquantes et doublons

5.3.1 Valeurs manquantes

On distingue trois mécanismes. Une donnée est manquante **complètement au hasard** (MCAR) lorsque son absence ne dépend d'aucune variable ; **au hasard** (MAR) lorsqu'elle dépend d'autres variables observées ; **non au hasard** (MNAR) lorsqu'elle dépend de la valeur manquante elle-même. Dans ce jeu de données, l'absence de titre ou de texte résulte plutôt d'un champ facultatif laissé vide : c'est une information sur le comportement de la cliente, à ne pas effacer. Le Tableau 5.2 compare les stratégies possibles.

__Tableau 5.2__ — Stratégies de traitement des valeurs manquantes

Stratégie	Quand l'utiliser	Inconvénient
Supprimer les lignes	Très peu de lignes concernées, absence aléatoire	Perte d'information ; biais si l'absence est informative
Imputer par la médiane ou le mode	Variables numériques ou catégorielles à faible taux de manquants	Réduit la variance ; statistique à apprendre sur l'entraînement seul
Créer une modalité « Unknown »	Variables catégorielles dont l'absence est informative	Ajoute une modalité
Remplacer par un texte vide	Champs textuels facultatifs	L'avis est conservé mais sans contenu textuel

Le projet combine deux niveaux de traitement. Dans le notebook 01, les chaînes vides sont converties en modalité « Unknown » et les valeurs textuelles et catégorielles manquantes sont traitées. Dans le notebook 02, un `SimpleImputer` (médiane pour les variables numériques, valeur la plus fréquente pour les variables catégorielles) sécurise le traitement de toute valeur manquante restante, avec des statistiques apprises sur le jeu d'entraînement uniquement (chapitre chapitre 6).

5.3.2 Doublons

Un doublon est une ligne strictement identique à une autre. Les doublons gonflent artificiellement certains effectifs et, surtout, peuvent se retrouver à la fois dans l'entraînement et dans le test, ce qui constitue une fuite de données (section section 2.8). Le nettoyage supprime donc les doublons exacts *avant* tout découpage.

5.4 Statistiques descriptives et distributions

Pour chaque variable numérique, le notebook calcule les résumés de tendance centrale (moyenne, médiane) et de dispersion (écart-type, quartiles, étendue). La **médiane** est préférée à la moyenne lorsque la distribution est asymétrique, car elle n'est pas tirée par les valeurs extrêmes. Les distributions sont visualisées par des histogrammes et par des courbes de densité estimées par noyau (KDE), qui lissent l'histogramme :

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right) \tag{5.1}$$

où K est un noyau (en général gaussien) et h la largeur de bande qui contrôle le lissage : trop petite, la courbe épouse le bruit ; trop grande, elle masque les modes. Trois lectures sont systématiques : la *forme* (symétrie, asymétrie, plusieurs modes), l'*étendue* (valeurs extrêmes plausibles ?) et la *discrétisation* (variables entières à peu de niveaux).

- **Age** : variable entière dont les valeurs observées vont de 18 à 99 ans ; les âges très élevés sont les candidats naturels à un examen de plausibilité.

- **Positive Feedback Count** : compteur dont les valeurs vont de 0 à 122. Un compteur de ce type est typiquement très asymétrique à droite : la plupart des avis reçoivent peu ou pas de retours, quelques-uns en reçoivent beaucoup.
- **Rating** : variable discrète à cinq niveaux. Les notes élevées dominent (dans les analyses publiées sur ce jeu, la note maximale représente plus de la moitié des avis), ce qui est cohérent avec les 82,2 % de recommandations.

5.5 Valeurs atypiques (règle de Tukey)

La règle de Tukey (section section 2.9.5) signale les observations situées hors de l'intervalle [Q_1 − 1,5 IQR ; Q_3 + 1,5 IQR]. Son application mécanique appelle des précautions :

- **Pour une variable à quelques niveaux comme la note**, la règle peut produire des résultats trompeurs. Les analyses publiées indiquent qu'environ 78 % des avis ont 4 ou 5 et qu'environ 10 % ont 1 ou 2 : les quartiles valent donc 4 et 5, l'IQR vaut 1 et la limite basse est 4 − 1,5 = 2,5. Les notes 1 et 2 seraient ainsi qualifiées d'« atypiques », alors qu'il s'agit d'avis parfaitement légitimes, et précisément des plus liés aux non-recommandations.
- **Pour un compteur asymétrique comme Positive Feedback Count**, les valeurs élevées sont atypiques par construction, mais correspondent à des avis réellement très populaires.

Le nettoyage décrit à la section section 5.8 ne comporte pas de suppression d'observations au titre des valeurs atypiques. Ce choix est cohérent : les valeurs signalées sont plausibles, la médiane et les quartiles utilisés par le pipeline sont peu sensibles aux extrêmes, et supprimer les notes basses retirerait justement l'information la plus utile pour repérer les non-recommandations.

5.6 Analyse de la cible et corrélations

La répartition de `Recommended IND` (82,2 % / 17,8 %, section section 3.5) est analysée en premier : elle fixe le plancher de performance du baseline Zero-R et impose le découpage stratifié. Le notebook calcule ensuite la **matrice de corrélation de Pearson** entre variables numériques (section section 2.9.4). Les repères usuels, proposés par Cohen, qualifient une corrélation de faible autour de 0,1, de modérée autour de 0,3 et de forte à partir de 0,5 en valeur absolue. La matrice sert à repérer d'éventuelles variables redondantes et les variables les plus liées à la cible. Comme l'âge, le nombre de retours positifs et la note ne sont pas des variables continues à distribution normale, ces coefficients restent des indicateurs descriptifs ; l'inférence est confiée aux tests de rangs et du χ^2 ci-dessous.

5.7 Tests statistiques et justification de leur choix

Le cahier des charges exige au moins deux tests statistiques scientifiquement appropriés et explicitement justifiés. Le projet en réalise deux, qui couvrent les deux grandes familles de situations : une variable ordinale contre une cible binaire, et deux variables qualitatives. Les deux tests sont exécutés avec la bibliothèque SciPy (`scipy.stats`) au seuil $\alpha = 0{,}05$.

5.7.1 Test de Mann-Whitney U : Rating × Recommended IND

- **Question.** Les clientes qui recommandent un article donnent-elles des notes différentes de celles qui ne le recommandent pas ?
- **Hypothèses.** H_0 : la distribution de Rating est la même dans les deux groupes (avis recommandés / non recommandés). H_1 : l'un des groupes tend à donner des notes plus élevées.

- **Pourquoi ce test.** (1) Rating est une variable *ordinale* à cinq niveaux : la moyenne et la variance n'ont pas de sens strict, ce qui exclut le test t de Student. (2) Sa distribution est très asymétrique, bornée à 5 (effet plafond) : l'hypothèse de normalité est manifestement fausse. (3) Il y a exactement deux groupes *indépendants* (chaque avis appartient à un seul groupe). (4) Avec plus de vingt mille observations, l'approximation normale du test est excellente, et la correction pour les très nombreux ex æquo (cinq valeurs seulement) est prévue.
- **Règle de décision.** On rejette H_0 si $p < 0{,}05$. La taille d'effet est la probabilité de supériorité $U_1/(n_1 n_2)$ (section section 2.9.2) : plus elle s'éloigne de 0,5, plus la note discrimine les groupes.
- **Lien avec la modélisation.** Rating fait partie des variables explicatives ; le test mesure donc aussi le pouvoir prédictif attendu de cette variable, et rappelle qu'une partie de la performance des modèles provient d'une variable fortement liée à la cible (section section 8.6).

5.7.2 Test du χ² d'indépendance : Department Name × Recommended IND

- **Question.** La probabilité de recommander dépend-elle du département du produit (robes, hauts, bas, vestes, lingerie, tendance) ?
- **Hypothèses.** H_0 : Department Name et Recommended IND sont indépendantes. H_1 : elles sont associées.
- **Pourquoi ce test.** (1) Les deux variables sont qualitatives (nominale à 6 modalités, binaire) : le tableau de contingence 6 × 2 est l'outil naturel, alors qu'une corrélation de Pearson n'aurait aucun sens pour une variable nominale. (2) Les effectifs (plus de 23 000 avis) rendent la condition « effectifs attendus ≥ 5 » vérifiable et très probablement satisfaite, ce qui rend le test exact de Fisher inutile. (3) Les observations sont indépendantes (un avis par ligne).
- **Règle de décision.** Le nombre de degrés de liberté est $(6 - 1)(2 - 1) = 5$ (6 si la modalité « Unknown » est conservée). On rejette H_0 si $p < 0{,}05$; la force de l'association se lit sur le V de Cramér.
- **Précaution.** Un résultat significatif établit une association, non une causalité, et n'indique pas quels départements s'écartent de l'indépendance : les résidus standardisés permettent de les repérer.

5.7.3 Synthèse des deux tests

__Tableau 5.3__ — Comparaison des deux tests statistiques du projet

	Mann-Whitney U	χ² d'indépendance
Variables	Rating (ordinale) × Recommended IND (binaire)	Department Name (nominale) × Recommended IND (binaire)
Nature du test	Non paramétrique, fondé sur les rangs	Association entre variables qualitatives
Hypothèse nulle	Mêmes distributions de notes dans les deux groupes	Indépendance des deux variables
Conditions	Groupes indépendants ; variable au moins ordinale	Observations indépendantes ; effectifs attendus ≥ 5
Taille d'effet	Probabilité de supériorité $U_1/(n_1 n_2)$	V de Cramér
Seuil de décision	$\alpha = 0{,}05$	$\alpha = 0{,}05$
Alternative écartée	Test t de Student (normalité non vérifiée, variable ordinale)	Corrélation de Pearson (variable nominale)

Ces deux tests ont pour objet de valider l'existence d'un lien entre ces variables et la cible ; leurs sorties détaillées (statistique U, statistique du χ², p-valeurs) sont affichées par le notebook 01. Avec deux tests, le risque cumulé d'erreur de type I reste modéré ; une correction de Bonferroni (seuil de 0,025 par test) ou de Benjamini-Hochberg serait possible si l'on multipliait les tests.

> **Significatif n'est pas synonyme d'important**
>
> Avec plus de vingt mille avis, presque n'importe quel écart devient statistiquement significatif. La lecture correcte associe toujours la p-valeur à une taille d'effet et à la plausibilité métier : c'est la force du lien, pas seulement son existence, qui justifie l'usage d'une variable dans le modèle.

5.8 Procédure de nettoyage

Le nettoyage applique les opérations suivantes, dans cet ordre :

1. **Suppression des doublons exacts**, pour éviter qu'un même avis figure à la fois dans l'entraînement et dans le test ;
2. **Suppression des espaces superflus** en début et en fin de chaîne : sans cela, « Tops » et « Tops » suivi d'un espace seraient comptés comme deux modalités distinctes ;
3. **Remplacement des chaînes vides par « Unknown »**, ce qui uniformise la représentation de l'absence d'information (valeur manquante, chaîne vide et chaîne d'espaces sont des écritures différentes d'une même absence) ;
4. **Traitement des valeurs textuelles et catégorielles manquantes** ;
5. **Suppression des lignes dont la cible est manquante**, puisqu'elles ne peuvent servir ni à l'apprentissage ni à l'évaluation ;
6. **Sauvegarde** du résultat dans `data/processed/reviews_clean.csv`.

Comme les doublons sont très rares (section section 3.4) et que la cible est complète, le jeu nettoyé compte pratiquement autant de lignes que le jeu brut ; le décompte exact est affiché par le notebook. Le jeu brut `reviews.csv` n'est jamais modifié : le nettoyage produit un nouveau fichier, ce qui préserve la traçabilité.

5.9 Bilan du chapitre

- Le jeu est de bonne qualité : la cible est complète, les valeurs manquantes se concentrent sur deux champs textuels facultatifs et les doublons sont très rares.
- Les valeurs atypiques sont examinées mais conservées, car elles sont plausibles et informatives.
- Deux tests complémentaires (Mann-Whitney U et χ^2) sont choisis en fonction de la nature des variables et justifiés explicitement.
- Le jeu nettoyé est prêt pour le prétraitement, qui fait l'objet du chapitre suivant.

6 Prétraitement et encodage des variables

Ce chapitre décrit le notebook `02_preprocessing_encoding.ipynb`. Son rôle est de transformer un tableau hétérogène (nombres, catégories, textes libres) en une matrice numérique que les algorithmes peuvent exploiter, **sans que le jeu de test n'influence ces transformations**.

6.1 Séparation X / y et suppression de Clothing ID

Le jeu nettoyé est d'abord séparé en une matrice de variables explicatives **X** et un vecteur cible **y** (`Recommended IND`). La colonne `Clothing ID` est supprimée de X. Elle est stockée sous forme d'entier, mais ce n'est pas une grandeur numérique au sens statistique : le produit n° 1000 n'est pas « plus grand » que le produit n° 500, et faire des moyennes d'identifiants n'a aucun sens. Trois arguments confirment ce choix :

- **Ordre et magnitude arbitraires.** Un algorithme à distance ou à marge (K-NN, SVM) interpréterait à tort l'écart entre deux identifiants comme une proximité entre produits.
- **Cardinalité élevée.** Avec 1 206 identifiants distincts pour 23 486 avis, soit environ 19 avis par produit en moyenne, un encodage one-hot ajouterait plus d'un millier de colonnes presque vides et favoriserait le surapprentissage sur des produits précis.
- **Absence de généralisation.** Un modèle fondé sur l'identifiant ne pourrait rien dire d'un nouveau produit ; on souhaite au contraire qu'il s'appuie sur ce que dit la cliente et sur le type de produit (division, département, catégorie).

Après cette étape, X contient huit colonnes : trois numériques (Age, Rating, Positive Feedback Count), trois catégorielles (Division Name, Department Name, Class Name) et deux textuelles (Title, Review Text).

6.2 Découpage entraînement / test stratifié

Le jeu est découpé en un jeu d'**entraînement** (80 %) et un jeu de **test** (20 %). Pour un jeu nettoyé d'environ 23 500 avis, cela représente de l'ordre de 18 800 avis pour l'entraînement et de 4 700 pour le test. Le découpage est **stratifié** (`stratify=y`) : chaque sous-ensemble conserve les proportions de classes du jeu complet (Figure 6.1). La conséquence se vérifie dans les résultats : l'accuracy du baseline Zero-R sur le jeu de test (0,8223) coïncide avec la proportion de la classe 1.

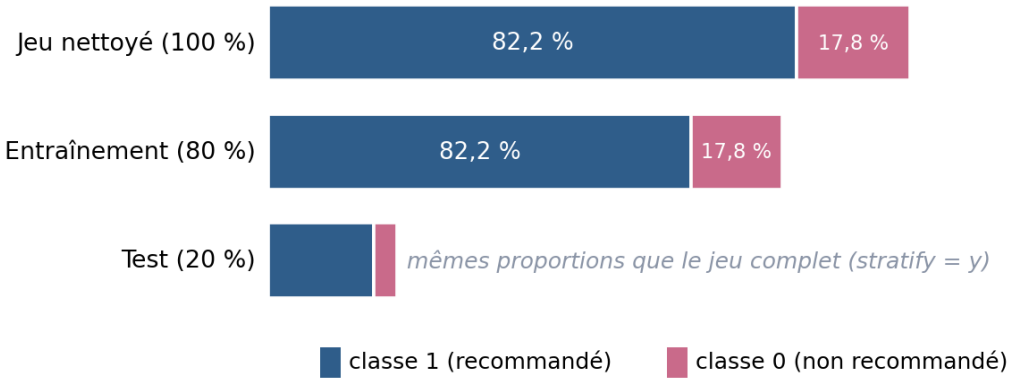


***Figure 6.1** — Découpage stratifié 80 % / 20 % : les proportions de classes sont identiques dans les trois ensembles.*

Le choix 80/20 est un compromis courant : le jeu d'entraînement reste assez grand pour apprendre un vocabulaire riche, et le jeu de test, de plus de 4 000 avis, est assez grand pour estimer l'accuracy avec une

incertitude d'environ 0,4 point (section section 8.3). Le découpage est réalisé *avant* toute transformation, condition indispensable pour que le préprocesseur soit ajusté sur l'entraînement seul.

6.3 Le ColumnTransformer

Les trois familles de variables n'appellent pas le même traitement. Le `ColumnTransformer` de scikit-learn applique à chaque groupe de colonnes sa propre chaîne de transformations, puis assemble les résultats en une seule matrice (Figure 6.2 et Tableau 6.1).

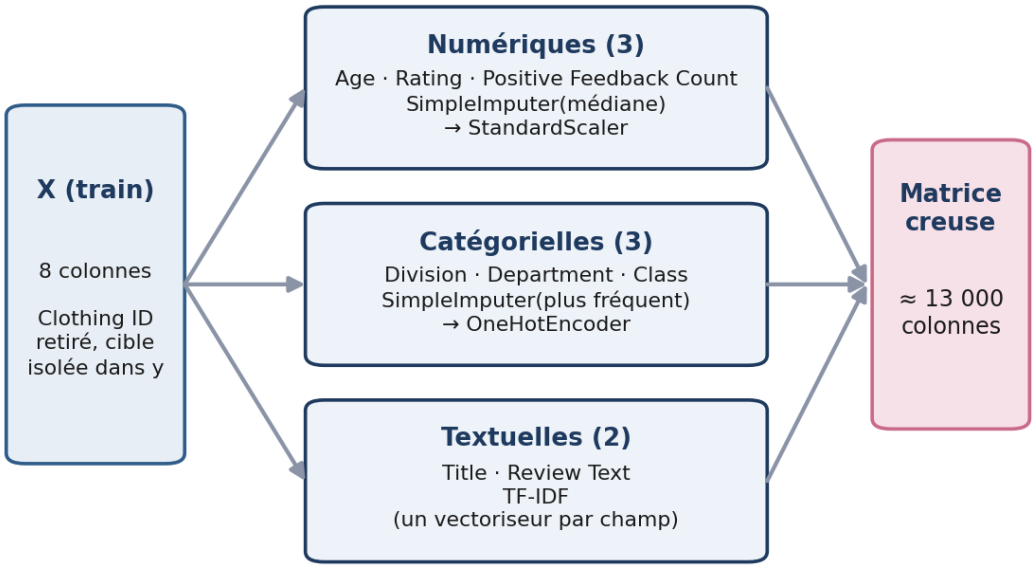


***Figure 6.2** — Structure du ColumnTransformer : trois branches parallèles assemblées en une matrice creuse.*

***Tableau 6.1** — Traitement appliqué à chaque famille de variables*

Famille	Variables	Transformation	Justification
Numériques	Age, Rating, Positive Feedback Count	`SimpleImputer(median)` puis `StandardScaler`	Médiane robuste aux extrêmes ; mise à l'échelle pour les modèles à distance ou à marge
Catégorielles	Division, Department, Class	`SimpleImputer(most_frequent)` puis `OneHotEncoder`	Aucun ordre artificiel entre modalités
Textuelles	Title, Review Text	TF-IDF, un vectoriseur par champ	Transforme le texte en poids numériques tenant compte de la rareté des termes

6.3.1 Variables numériques

Les valeurs manquantes éventuelles sont remplacées par la médiane de l'entraînement, puis chaque variable est standardisée par son score z (équation équation (2.11)). Les jeux Age, Rating et Positive Feedback Count n'ayant aucune valeur manquante dans le jeu brut, l'imputeur joue ici un rôle de *garde-fou* : il rend le pipeline robuste à une donnée manquante saisie ultérieurement, par exemple dans l'application. La standardisation est essentielle pour le K-NN et le SVM, qui comparent des distances ou cherchent une marge : sans elle, l'âge (dizaines) dominerait la note (1 à 5).

6.3.2 Variables catégorielles

Les valeurs manquantes sont remplacées par la modalité la plus fréquente, puis chaque modalité devient une colonne binaire. Avec 3 modalités pour Division Name, 6 pour Department Name et 20 pour Class Name, l'encodage produit au plus 3 + 6 + 20 = **29 colonnes binaires**, auxquelles s'ajoutent les éventuelles modalités « Unknown » créées au nettoyage. Ces trois variables sont emboîtées (une classe appartient à un département, qui appartient à une division) : elles sont partiellement redondantes, ce qui est sans gravité pour les modèles retenus.

6.3.3 Variables textuelles

Le titre et le corps de l'avis sont vectorisés **séparément**, chacun par son propre vectoriseur TF-IDF (section section 2.6). Ce choix se justifie par la nature différente des deux champs : le titre est court et synthétique, le corps est long et détaillé ; un même mot n'a ni la même fréquence documentaire ni le même poids dans les deux. Vectoriser séparément conserve cette information, alors qu'une vectorisation du texte concaténé la mélangerait. Le vocabulaire et les idf sont appris sur l'entraînement.

Au total, la matrice obtenue compte environ **13 000 colonnes**. Comme les parties numérique et catégorielle ne représentent que 3 + 29 = 32 colonnes au plus, la quasi-totalité de ces colonnes provient du TF-IDF : l'information textuelle domine donc largement l'espace de représentation.

6.4 Absence de fuite de données

Le projet applique strictement la règle : `fit_transform` sur le jeu d'entraînement, `transform` seul sur le jeu de test. Le Tableau 6.2 indique, pour chaque étape, ce qui est appris sur l'entraînement et la façon dont le test en bénéficie sans l'influencer.

__Tableau 6.2__ — Ce que le préprocesseur apprend sur l'entraînement et la façon dont il traite le test

Étape	Appris sur l'entraînement	Appliqué au test
Imputation numérique	Médiane de chaque variable	Les valeurs manquantes reçoivent ces médianes
Standardisation	Moyenne μ et écart-type σ	Score z calculé avec les μ et σ de l'entraînement
Imputation catégorielle	Modalité la plus fréquente	Les valeurs manquantes reçoivent cette modalité
Encodage one-hot	Liste des modalités	Mêmes colonnes ; une modalité jamais vue ne peut pas recevoir de colonne propre
TF-IDF	Vocabulaire et idf de chaque terme	Poids calculés avec le vocabulaire et les idf de l'entraînement ; mots inconnus ignorés

> **Une nuance méthodologique**
>
> D'après la chaîne de traitement décrite (le notebook 03 prend en entrée les matrices déjà transformées), le préprocesseur est ajusté sur l'ensemble du jeu d'entraînement avant la validation croisée. Les plis de validation ont donc contribué au calcul des idf et des moyennes. Cette fuite est mineure, car elle reste interne à l'estimation par validation croisée et **n'atteint pas le jeu de test**, dont les résultats restent fiables. Elle pourrait être supprimée en plaçant le préprocesseur dans un `Pipeline` passé à `GridSearchCV` (chapitre chapitre 10).

6.5 Matrices creuses et coût mémoire

Une matrice dense de 18 800 lignes et 13 000 colonnes stockée en nombres à virgule flottante sur 8 octets occuperait 18 800 × 13 000 × 8 octets, soit près de **2,0 Go**. Or un avis ne contient qu'une cinquantaine de termes distincts : la matrice TF-IDF est creuse. En supposant 50 valeurs non nulles par ligne et un stockage CSR (8 octets par valeur plus 4 octets par indice de colonne), la matrice creuse occupe seulement environ **11 Mo**, soit un facteur 170 de gain (estimation à titre d'ordre de grandeur). C'est ce qui rend possible l'entraînement de tous les modèles sur un ordinateur personnel, et ce qui explique pourquoi le choix de modèles acceptant des entrées creuses (section section 7.2) est déterminant.

6.6 Artefacts sérialisés

Les objets entraînés sont sauvegardés avec `joblib` dans le dossier `models/` afin d'être rechargés par l'application sans ré-entraînement (Tableau 6.3).

__Tableau 6.3__ — Fichiers sérialisés du dossier models/

Fichier	Contenu	Produit par	Utilisé par
`preprocessor.pkl`	ColumnTransformer ajusté (imputation, scaling, one-hot, TF-IDF)	Notebook 02	Notebook 03, application
`review_tfidf.pkl`	Vectoriseur TF-IDF du champ Review Text	Notebook 02	Archivage du vectoriseur
`title_tfidf.pkl`	Vectoriseur TF-IDF du champ Title	Notebook 02	Archivage du vectoriseur
`best_model.pkl`	Modèle final : Linear SVC optimisé	Notebook 03	Application

Deux précautions s'imposent. D'une part, un fichier `.pkl` n'est lisible de façon fiable qu'avec la version de scikit-learn qui l'a produit : c'est pourquoi `requirements.txt` fige les versions. D'autre part, charger un fichier *pickle* exécute du code : on ne doit charger que des fichiers dont on connaît l'origine, comme ceux générés par les notebooks du projet.

6.7 Bilan du chapitre

- Clothing ID est retiré : c'est un identifiant, non une grandeur statistique.
- Le découpage 80/20 est stratifié et précède toute transformation.
- Trois branches de transformation adaptées aux types de variables produisent une matrice creuse d'environ 13 000 colonnes.
- Le préprocesseur est ajusté sur l'entraînement seul ; le jeu de test reste vierge de toute influence.

7 Modélisation et optimisation des hyperparamètres

Ce chapitre décrit le notebook `03_classification_models.ipynb`. Il présente le protocole expérimental, la théorie de chacun des six algorithmes comparés (principe, formules, hyperparamètres, atouts et limites sur des données textuelles creuses), la recherche systématique des hyperparamètres et la sélection du modèle final.

7.1 Protocole expérimental

Tous les modèles sont entraînés et évalués dans les mêmes conditions, ce qui garantit la comparabilité des résultats :

1. les six approches reçoivent les **mêmes matrices** d'entraînement et de test, issues du notebook 02 ;
2. chaque modèle réglable est optimisé par `GridSearchCV` avec **validation croisée stratifiée** sur le seul jeu d'entraînement (le cahier des charges impose K ≥ 5 plis) ;
3. le meilleur estimateur de chaque recherche est ré-entraîné sur tout l'entraînement, puis **évalué une seule fois** sur le jeu de test avec trois métriques : accuracy, F1-score (classe 1) et ROC-AUC ;
4. une graine aléatoire `random_state=42` est fixée pour les modèles à composante aléatoire (Decision Tree, Random Forest, Linear SVC) ;
5. le modèle le plus performant est sérialisé dans `models/best_model.pkl`.

__Tableau 7.1__ — Les six approches comparées

Modèle	Famille	Hyperparamètres à régler	Optimisé par GridSearchCV
Zero-R	Baseline	Aucun	Non (exempté)
K-Nearest Neighbors	À base d'instances	Oui	Oui
Decision Tree	Arbre de décision	Oui	Oui
Random Forest	Ensemble d'arbres (bagging)	Oui	Oui
Multinomial Naïve Bayes	Probabiliste (Bayes)	Oui (lissage)	Oui
Linear SVC	Marge maximale (SVM linéaire)	Oui (C)	Oui

7.2 Les algorithmes comparés

7.2.1 Zero-R : le baseline

Le classifieur **Zero-R** (règle zéro) ignore toutes les variables explicatives et prédit toujours la classe majoritaire de l'entraînement, ici la classe 1. Son accuracy attendue est donc égale à la proportion de cette classe : 0,8223 sur le jeu de test du projet. Il détecte tous les avis recommandés (rappel de 1) mais aucun avis non recommandé (spécificité de 0), et son F1-score vaut $2p/(1 + p) = 0{,}9025$. Il n'a aucun hyperparamètre et n'est donc pas optimisé. Son rôle est de fixer un **plancher** : un modèle qui ne le dépasse pas n'apporte aucune information. L'AUC d'un score constant vaut 0,5 ; elle n'est pas rapportée pour ce modèle (« N/A » dans le tableau des résultats).

7.2.2 K-Nearest Neighbors (K-NN)

Le K-NN (Cover et Hart, 1967) est un algorithme *à base d'instances* : il n'apprend aucun modèle explicite. Pour classer un nouvel avis x, il recherche les K avis d'entraînement les plus proches selon une distance et prédit la classe majoritaire parmi eux (Figure 7.1), éventuellement en pondérant chaque voisin par l'inverse de sa distance. Les distances usuelles sont la distance euclidienne, la distance de Manhattan et la distance cosinus :

$$d_2(x,x') = \sqrt{\sum_j (x_j - x'_j)^2} \qquad d_1(x,x') = \sum_j |x_j - x'_j| \tag{7.1}$$

$$d_{\cos}(x,x') = 1 - \frac{x \cdot x'}{\|x\| \, \|x'\|} \tag{7.2}$$

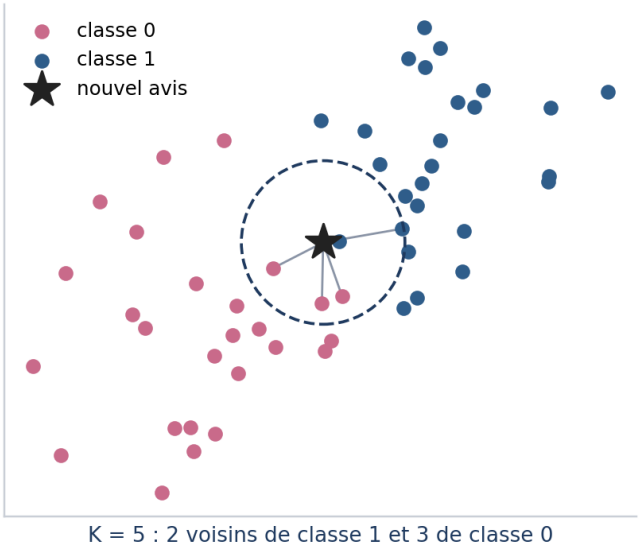


Figure 7.1 *— Principe du K-NN avec K = 5 : le nouvel avis (étoile) est classé selon la majorité de ses cinq plus proches voisins (données synthétiques).*

- **Hyperparamètres.** Le nombre de voisins K, la pondération des voisins et la métrique de distance. Un K petit donne un modèle très flexible (variance élevée, surapprentissage) ; un K grand lisse la frontière (biais élevé).
- **Mise à l'échelle indispensable.** Les distances mélangent toutes les colonnes : sans standardisation, une variable de grande amplitude écrase les autres.
- **Malédiction de la dimension.** Dans un espace de plus de dix mille colonnes, les distances entre points tendent à se ressembler, et la notion de « plus proche voisin » perd de son pouvoir discriminant. Pour des vecteurs TF-IDF normalisés en norme L2, la distance euclidienne au carré vaut $2(1 - \cos\theta)$: distance euclidienne et distance cosinus classent alors les voisins de la même façon.
- **Coût.** Il n'y a pas d'entraînement, mais chaque prédiction exige de comparer l'avis à tous ceux de l'entraînement, ce qui est coûteux pour un grand jeu et de grandes dimensions.

7.2.3 Arbre de décision (Decision Tree)

L'arbre de décision CART (Breiman et al., 1984) partitionne récursivement l'espace des variables par des tests simples (« le terme *perfect* a-t-il un poids supérieur à un seuil ? »). À chaque nœud, l'algorithme choisit la variable et le seuil qui réduisent le plus l'**impureté** des nœuds enfants. Pour un nœud t contenant la proportion p_k d'individus de la classe k, deux mesures d'impureté sont courantes, l'indice de Gini et l'entropie :

$$G(t) = 1 - \sum_k p_k^2 \qquad H(t) = -\sum_k p_k \log_2 p_k \tag{7.3}$$

$$\Delta I = I(t) - \sum_{c \in \{\text{gauche, droite}\}} \frac{n_c}{n_t} I(c) \tag{7.4}$$

Le gain d'information ΔI mesure la baisse d'impureté obtenue par un découpage, I désignant G ou H. À titre d'illustration, avec les proportions 82,2 % / 17,8 % du projet, le nœud racine a pour impureté de Gini 0,29 et pour entropie 0,68 bit (Figure 7.2) ; une feuille parfaitement pure aurait une impureté nulle. La prédiction d'une feuille est la classe majoritaire de ses individus.

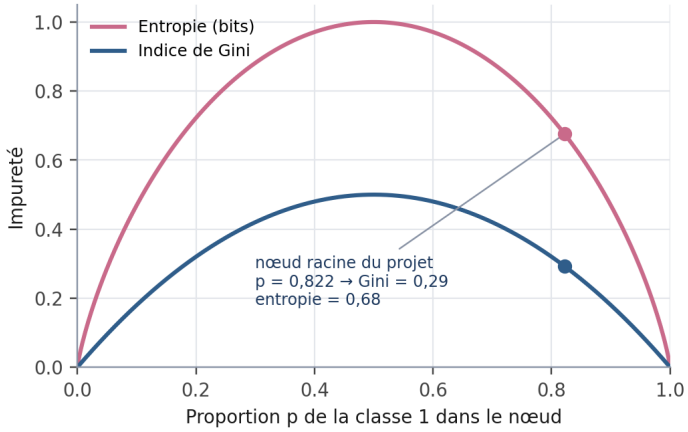


***Figure 7.2** — Indice de Gini et entropie d'un nœud binaire en fonction de la proportion p de la classe 1 ; le point correspond au nœud racine du projet.*

- **Hyperparamètres.** Profondeur maximale, nombre minimal d'individus par feuille ou par découpage, critère (Gini ou entropie), paramètre d'élagage. Un arbre trop profond mémorise l'entraînement ; l'*élagage* limite sa complexité.
- **Atouts.** Interprétabilité (les règles se lisent), insensibilité à l'échelle des variables, acceptation des matrices creuses.
- **Limites.** Forte variance (un petit changement des données peut produire un arbre très différent). Face à des milliers de variables textuelles faiblement informatives, chaque découpage n'utilise qu'un seul terme : un arbre unique combine mal beaucoup de signaux faibles, ce qui explique son retrait relatif dans les résultats.

7.2.4 Forêt aléatoire (Random Forest)

La forêt aléatoire (Breiman, 2001) combine B arbres entraînés chacun sur un échantillon bootstrap du jeu d'entraînement (tirage avec remise, ce qui retient environ 63 % d'individus distincts) et, à chaque nœud, sur un sous-ensemble aléatoire de variables (`max_features`). La prédiction est le vote majoritaire des arbres. L'agrégation réduit la variance : si les arbres ont chacun une variance σ^2 et une corrélation moyenne ρ, la variance de leur moyenne vaut

$$\mathrm{Var}(\bar{f}_B) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2 \tag{7.5}$$

Lorsque B est grand, le second terme disparaît : seule compte la corrélation ρ entre arbres, que le tirage aléatoire des variables sert précisément à diminuer. Les individus laissés de côté par chaque bootstrap (environ 37 %) fournissent une estimation interne de l'erreur dite *out-of-bag*.

- **Hyperparamètres.** Nombre d'arbres, profondeur maximale, nombre de variables considérées par nœud, taille minimale des feuilles.
- **Atouts.** Très robuste, peu sensible au surapprentissage, fournit des importances de variables, accepte les matrices creuses.
- **Limites.** Modèle volumineux (plusieurs dizaines à centaines d'arbres) et plus lent à entraîner et à interroger qu'un modèle linéaire ; sur du texte, où l'information est répartie sur de très nombreux termes faibles, il est en général moins bien adapté qu'un modèle linéaire.

7.2.5 Naïve Bayes multinomial

Le classifieur de Bayes naïf applique le théorème de Bayes en supposant les variables **conditionnellement indépendantes** sachant la classe. Dans sa version multinomiale (McCallum et Nigam, 1998), un document d est décrit par les effectifs $n_{w,d}$ de chaque mot w, et la classe prédite maximise la log-vraisemblance :

$$\hat{c} = \arg\max_{c} \left[\log P(c) + \sum_{w \in V} n_{w,d} \log P(w \mid c) \right] \tag{7.6}$$

Les probabilités P(w | c) sont estimées sur l'entraînement avec un **lissage de Laplace** (ou de Lidstone) de paramètre α, qui évite qu'un mot absent d'une classe ne reçoive une probabilité nulle et n'annule tout le produit :

$$P(w \mid c) = \frac{N_{w,c} + \alpha}{N_c + \alpha |V|} \tag{7.7}$$

où $N_{w,c}$ est le poids total du mot w dans les documents de la classe c, N_c le poids total de la classe et |V| la taille du vocabulaire. Les poids TF-IDF, bien que non entiers, jouent très bien le rôle d'effectifs dans cette formule.

- **Hyperparamètre.** Le paramètre de lissage α : proche de 0, le modèle colle aux données ; grand, il tire toutes les probabilités vers l'uniforme.
- **Atouts.** Extrêmement rapide, peu gourmand en données, bon point de départ pour la classification de texte.
- **Limites.** L'hypothèse d'indépendance est irréaliste (les mots d'un avis sont corrélés) et les probabilités produites sont mal calibrées. Le modèle exige aussi des **entrées positives** : la représentation qui lui est fournie doit être compatible avec cette contrainte, alors que les colonnes standardisées contiennent des valeurs négatives.

7.2.6 SVM linéaire (Linear SVC)

Le séparateur à vaste marge (SVM, Cortes et Vapnik, 1995) cherche l'hyperplan $w \cdot x + b = 0$ qui sépare les deux classes avec la **marge** la plus grande, c'est-à-dire la plus grande distance entre l'hyperplan et les points les plus proches (les *vecteurs de support*, Figure 7.3). La marge vaut $2/\|w\|$. Lorsque les classes ne sont pas parfaitement séparables, la version à marge souple autorise des erreurs mesurées par des variables d'écart ξ_i, pénalisées avec un poids C (les étiquettes sont notées $y_i \in \{-1, +1\}$) :

$$\min_{w, b, \xi} \tfrac{1}{2}\|w\|^2 + C \sum_{i=1}^{n} \xi_i \quad \text{avec} \quad y_i (w \cdot x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0 \tag{7.8}$$

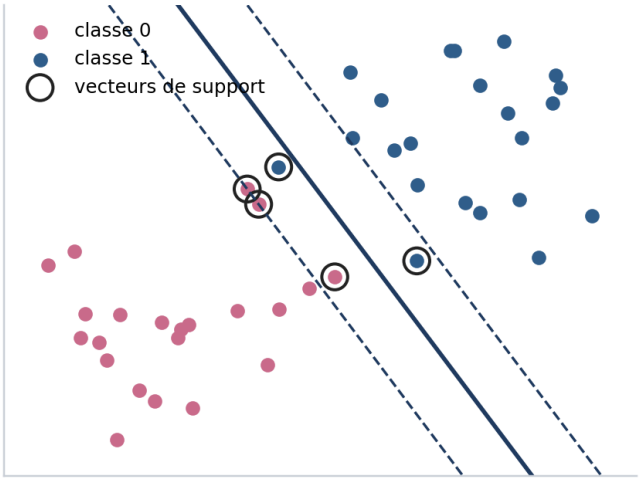


***Figure 7.3** — Principe du SVM linéaire : hyperplan séparateur (trait plein), marges (pointillés) et vecteurs de support entourés (données synthétiques).*

Un grand C punit sévèrement les erreurs (marge étroite, risque de surapprentissage) ; un petit C tolère davantage d'erreurs au profit d'une marge large (modèle plus régularisé). La prédiction repose sur la fonction de décision f(x) = w·x + b, dont le signe donne la classe et dont la valeur absolue mesure l'éloignement de l'hyperplan.

L'implémentation `LinearSVC` de scikit-learn s'appuie sur la bibliothèque LIBLINEAR (Fan et al., 2008), conçue pour les problèmes linéaires de grande taille. Elle utilise par défaut une pénalité L2 et la perte « hinge au carré » (`squared_hinge`) ; son coût croît à peu près linéairement avec le nombre de valeurs non nulles de la matrice.

- **Pourquoi il convient au texte** (Joachims, 1998). Les représentations TF-IDF vivent dans des espaces de très grande dimension, où les classes sont souvent quasi séparables par un hyperplan ; la plupart des termes sont faiblement mais réellement informatifs, et la matrice est creuse. Un modèle linéaire régularisé tire parti de tous ces signaux simultanément.
- **Noyaux non linéaires.** Les noyaux RBF ou polynomial (`SVC`) ont un coût au moins quadratique en nombre d'échantillons et apportent peu quand les données sont déjà quasi séparables : le noyau linéaire est le choix standard pour du texte.
- **Limite.** `LinearSVC` ne fournit pas de probabilités calibrées (pas de `predict_proba`) : on dispose d'un score de décision, qu'une étape de calibration (`CalibratedClassifierCV`) peut convertir en probabilités.

7.3 Recherche des hyperparamètres avec GridSearchCV

La procédure décrite à la section section 2.3 est appliquée à chaque modèle réglable : pour chaque combinaison de la grille, une validation croisée stratifiée sur l'entraînement calcule un score moyen ; la meilleure combinaison est retenue, puis le modèle est ré-entraîné sur tout l'entraînement. Le Tableau 7.2 liste les hyperparamètres principaux de chaque algorithme, c'est-à-dire ceux sur lesquels porte la recherche, et leur effet.

Tableau 7.2 — Hyperparamètres principaux de chaque algorithme

Algorithme	Hyperparamètres principaux	Effet sur le modèle
K-NN	`n_neighbors`, `weights`, métrique de distance	K petit : flexible et instable ; K grand : lissé. Pondération par la distance : voisins proches favorisés
Decision Tree	`max_depth`, `min_samples_leaf`, `criterion`	Profondeur et taille des feuilles contrôlent le surapprentissage ; critère Gini ou entropie
Random Forest	`n_estimators`, `max_depth`, `max_features`	Plus d'arbres : variance réduite, coût accru ; max_features règle la décorrélation des arbres
Multinomial NB	`alpha`	Intensité du lissage de Laplace
Linear SVC	`C`	Compromis entre largeur de marge et erreurs tolérées

Les grilles exactes explorées et les valeurs optimales retenues (attribut `best_params_` de chaque recherche) sont consignées dans le notebook 03. Trois remarques de méthode complètent le protocole :

- **Validation stratifiée.** Avec 17,8 % de classe minoritaire, la stratification des plis garantit une estimation stable du score.
- **Coût.** Le temps de calcul est proportionnel à |grille| × K ; la forêt aléatoire, qui ajuste de nombreux arbres sur plus de dix mille colonnes, est en général le modèle le plus coûteux à régler, alors que le Naïve Bayes et le Linear SVC sont nettement plus rapides.
- **Étanchéité du test.** Le jeu de test n'intervient à aucun moment de la recherche (section section 2.3).

7.4 Sélection du modèle final

Le **Linear SVC** est retenu parce qu'il obtient les meilleurs scores sur les trois métriques (chapitre chapitre 8), mais aussi pour des raisons pratiques : il est léger (un vecteur d'environ 13 000 coefficients), rapide à interroger, bien adapté aux matrices creuses de grande dimension et sa régularisation limite le surapprentissage. Il est sérialisé avec `joblib` dans `models/best_model.pkl`, et le préprocesseur correspondant dans `models/preprocessor.pkl`. Le couple modèle-préprocesseur est indissociable : un modèle ne peut recevoir que des données transformées par le préprocesseur avec lequel il a été entraîné.

7.5 Synthèse comparative des algorithmes

Tableau 7.3 — Synthèse des six approches

Algorithme	Principe	Atouts	Limites	Rang (F1)
Linear SVC	Hyperplan de marge maximale	Adapté au texte creux ; léger ; régularisé	Pas de probabilités calibrées	1
Random Forest	Vote de nombreux arbres décorrélés	Robuste ; importances de variables	Volumineux ; lent ; moins adapté au texte	2
Decision Tree	Découpages successifs de l'espace	Lisible ; insensible à l'échelle	Forte variance ; un terme par découpage	3
K-NN	Vote des K plus proches voisins	Aucun entraînement ; simple	Dimension élevée ; prédiction coûteuse	4
Multinomial NB	Bayes avec indépendance des mots	Très rapide ; bon baseline texte	Indépendance irréaliste ; entrées positives	5
Zero-R	Toujours la classe majoritaire	Plancher de référence	Aucune information exploitée	6

8 Résultats et évaluation

Ce chapitre présente les performances des six approches sur le jeu de test, les interprète avec la prudence que commande l'incertitude d'échantillonnage, analyse la structure des erreurs par classe et positionne le résultat par rapport à des références externes. Tous les chiffres de performance proviennent du README du dépôt ; les valeurs « reconstituées » sont déduites de ces chiffres par un calcul exact présenté à la section section 8.4.

8.1 Choix des métriques

Avec 82,2 % d'avis dans la classe majoritaire, l'accuracy seule est trompeuse : le baseline Zero-R atteint 0,8223 sans rien apprendre. Le projet privilégie donc deux métriques complémentaires. Le **F1-score** de la classe recommandée résume le compromis précision-rappel ; la **ROC-AUC** évalue la qualité du classement des avis par le score du modèle, indépendamment du seuil de décision et du déséquilibre (section section 2.5). L'accuracy est conservée comme repère lisible. Elle est complétée plus loin par les rappels de chaque classe.

8.2 Résultats comparatifs sur le jeu de test

__Tableau 8.1__ — Performances des modèles sur le jeu de test (source : README du dépôt)

Rang	Modèle	Accuracy	F1-score	ROC-AUC
1	**Linear SVC**	**0,9378**	**0,9622**	**0,9775**
2	Random Forest	0,9356	0,9613	0,9730
3	Decision Tree	0,9301	0,9563	0,9684
4	K-NN	0,9186	0,9519	0,9642
5	Multinomial NB	0,9052	0,9425	0,9531
6	Zero-R	0,8223	0,9025	N/A

Le Linear SVC arrive en tête sur les trois métriques, suivi de la forêt aléatoire, de l'arbre de décision, du K-NN et du Naïve Bayes multinomial. Le baseline Zero-R ferme la marche ; son AUC n'est pas rapportée (« N/A »).

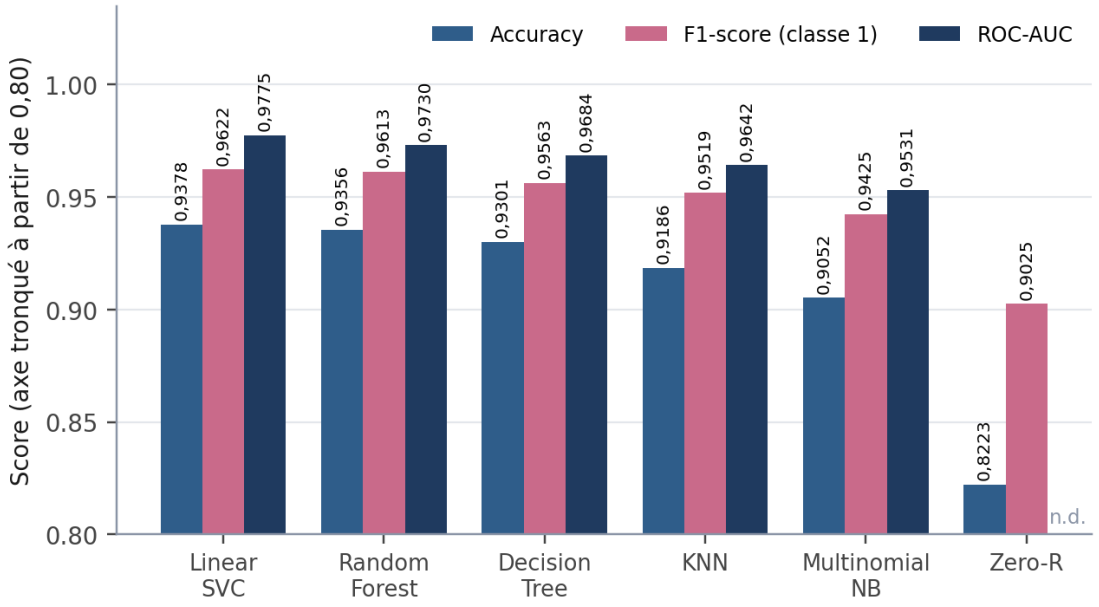


Figure 8.1 — Comparaison des modèles sur le jeu de test : accuracy, F1-score et ROC-AUC (axe tronqué à partir de 0,80).

8.3 Lecture des résultats

8.3.1 Apport par rapport au baseline

Tous les modèles dépassent nettement le baseline Zero-R, ce qui montre que le texte et les métadonnées portent un véritable signal prédictif. L'accuracy gagne entre 8 et 11,5 points. Le gain en F1 paraît plus modeste (de 0,04 à 0,06) parce que le baseline part déjà d'un F1 élevé de 0,9025, ce qui limite la marge de progression à 0,0975. Une lecture plus parlante est la **réduction relative du taux d'erreur** : le Linear SVC passe d'un taux d'erreur de 17,77 % (baseline) à 6,22 %, soit une réduction de 65 % (Tableau 8.2).

Tableau 8.2 — Apport de chaque modèle par rapport au baseline Zero-R (accuracy 0,8223 ; F1 0,9025)

Modèle	Gain d'accuracy (points)	Gain de F1	Taux d'erreur	Réduction relative de l'erreur
Linear SVC	+11,55	+0,0597	6,22 %	65 %
Random Forest	+11,33	+0,0588	6,44 %	64 %
Decision Tree	+10,78	+0,0538	6,99 %	61 %
K-NN	+9,63	+0,0494	8,14 %	54 %
Multinomial NB	+8,29	+0,0400	9,48 %	47 %

8.3.2 Écarts entre modèles et incertitude d'échantillonnage

Une accuracy mesurée sur un jeu de test fini est une *estimation*. Son erreur-type vaut approximativement

$$\mathrm{SE} = \sqrt{\frac{\hat{p}\,(1-\hat{p})}{n}} \tag{8.1}$$

soit, pour p = 0,9378 et un jeu de test d'environ 4 700 avis, 0,35 point d'accuracy (intervalle de confiance à 95 % d'environ ± 0,7 point). Le Tableau 8.3 exprime les écarts avec le Linear SVC en nombre d'erreurs-types.

Tableau 8.3 — Écarts avec le Linear SVC exprimés en erreurs-types (jeu de test supposé de 4 700 avis)

Modèle comparé au Linear SVC	Écart d'accuracy (points)	En erreurs-types	Lecture
Random Forest	0,22	0,6	Écart non distinguable du bruit d'échantillonnage
Decision Tree	0,77	2,2	Écart d'environ deux erreurs-types : probable mais à confirmer
K-NN	1,92	5,5	Écart net
Multinomial NB	3,26	9,3	Écart très net

L'écart entre le Linear SVC et la forêt aléatoire (0,22 point d'accuracy, 0,0009 de F1, 0,0045 d'AUC) est inférieur à une erreur-type : sur ces seules données, les deux modèles sont statistiquement indiscernables. Le choix du Linear SVC s'appuie donc moins sur cet écart que sur son AUC légèrement supérieure et sur ses avantages pratiques (légèreté, rapidité, adaptation aux matrices creuses). Une précision supplémentaire s'impose : les deux modèles étant évalués sur *les mêmes* avis, la variance de leur différence est plus faible que ne le suggère ce calcul ; le test de McNemar, qui exploite cet appariement, serait l'outil adapté (chapitre chapitre 10).

8.3.3 Interprétation par famille de modèles

- **Forêt aléatoire contre arbre unique.** L'agrégation de nombreux arbres gagne 0,55 point d'accuracy sur l'arbre seul : c'est la réduction de variance attendue (équation équation (7.5)).
- **K-NN.** Il se classe quatrième : dans un espace de plus de dix mille dimensions, les distances perdent leur pouvoir discriminant (section section 7.2).
- **Naïve Bayes multinomial.** Il reste le plus faible des modèles appris, mais conserve une avance de plus de 8 points d'accuracy sur le baseline : l'hypothèse d'indépendance des mots est irréaliste, sans être rédhibitoire pour cette tâche.
- **Qualité du classement (AUC).** Toutes les AUC dépassent 0,95. Pour le Linear SVC, une AUC de 0,9775 signifie qu'un avis recommandé tiré au hasard reçoit un meilleur score qu'un avis non recommandé tiré au hasard dans 97,75 % des paires.

8.4 Analyse par classe : reconstitution des matrices de confusion

Le README ne publie que l'accuracy, le F1-score et l'AUC. Ces trois nombres suffisent pourtant à reconstituer, *exactement* au sens de l'arithmétique, la structure des erreurs de chaque modèle, sous trois hypothèses vérifiables : (i) la classe positive est la classe 1 ; (ii) le F1 rapporté est le F1 binaire de la classe 1, ce que confirme le baseline (2p/(1 + p) = 0,9025) ; (iii) la proportion de la classe 1 dans le jeu de test est p = 0,8223, valeur de l'accuracy de Zero-R. Notons VP, FP, FN et VN les effectifs exprimés en fraction du jeu de test. Les deux équations A = VP + VN et F_1 = 2VP/(2VP + FP + FN), jointes aux relations VP + FN = p et VN + FP = 1 − p, donnent (démonstration en annexe B) :

$$VP = \frac{F_1(1-A)}{2(1-F_1)} \qquad FP = VP - (A - 1 + p) \qquad FN = p - VP \qquad VN = 1 - p - FP \tag{8.2}$$

Recalculées à partir de ces effectifs, l'accuracy et le F1 retrouvent par construction les valeurs du README. Comme les métriques publiées sont arrondies à quatre décimales, les valeurs reconstituées sont connues à environ **± 0,01** près ; elles sont donc données avec deux décimales et doivent être confirmées par le *classification report* et la matrice de confusion du notebook 03 lorsqu'on les dispose.

Tableau 8.4 — Métriques par classe reconstituées à partir de l'accuracy, du F1 et de p = 0,8223 (incertitude ± 0,01)

Modèle	Précision (cl. 1)	Rappel (cl. 1)	Spécificité (rappel cl. 0)	F1 (cl. 0)	Macro-F1	Accuracy équilibrée
Linear SVC	0,96	0,96	0,82	0,82	0,89	0,89
Random Forest	0,95	0,97	0,76	0,81	0,88	0,87
Decision Tree	0,98	0,93	0,93	0,83	0,89	0,93
K-NN	0,93	0,98	0,64	0,74	0,84	0,81
Multinomial NB	0,94	0,94	0,72	0,73	0,84	0,83
Zero-R	0,82	1,00	0,00	0,00	0,45	0,50

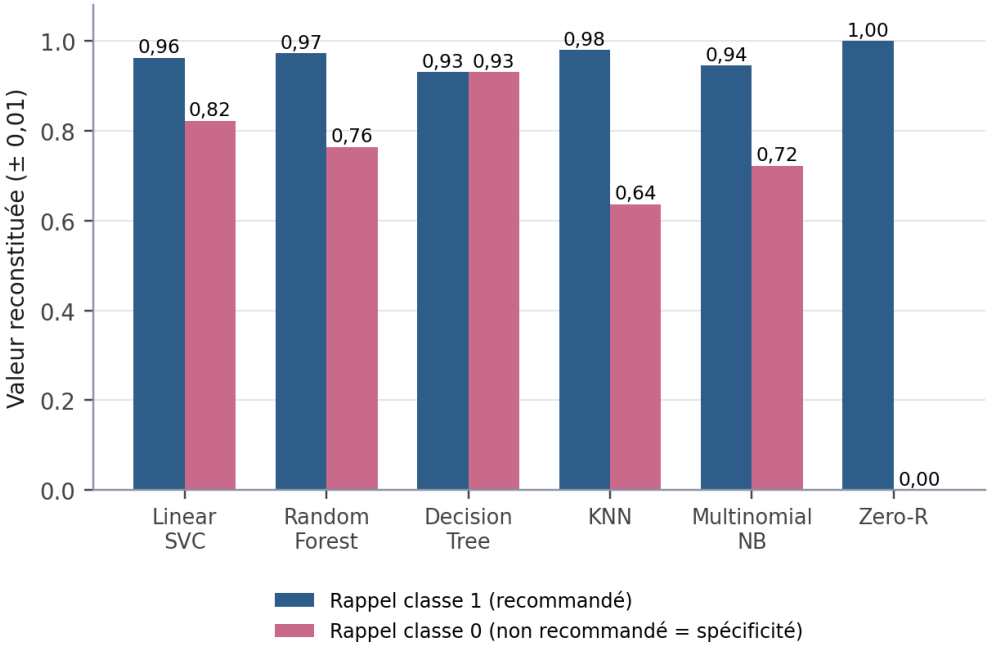


Figure 8.2 — Rappel de la classe 1 et rappel de la classe 0 (spécificité) de chaque modèle, valeurs reconstituées (± 0,01).

Pour le modèle retenu, la matrice de confusion reconstituée, exprimée en pourcentage du jeu de test, est la suivante (Figure 8.3). Sur un jeu d'environ 4 700 avis, cela représente de l'ordre de 150 avis recommandés classés à tort comme non recommandés (FN) et autant d'avis non recommandés classés à tort comme recommandés (FP).

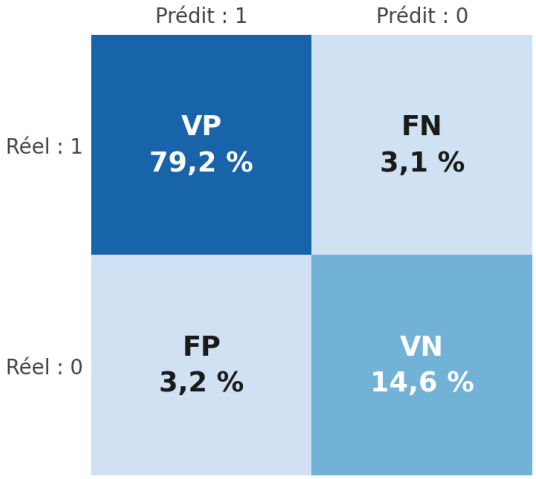


***Figure 8.3** — Matrice de confusion reconstituée du Linear SVC, en pourcentage du jeu de test (incertitude d'environ ± 0,2 point).*

8.4.1 Enseignements

- **Le Linear SVC offre le meilleur équilibre.** Précision et rappel de la classe 1 valent environ 0,96, et il détecte environ 82 % des avis non recommandés. Autrement dit, environ 18 % des avis non recommandés lui échappent : la classe minoritaire reste la plus difficile à repérer.
- **La forêt, le K-NN et le Naïve Bayes sont « optimistes ».** Leur rappel de la classe 1 est très élevé (0,97, 0,98 et 0,94) mais leur spécificité est nettement plus faible (0,76, 0,64 et 0,72) : ils tendent à prédire « recommandé » par défaut, effet typique du déséquilibre. Le K-NN est le cas extrême, avec seulement 64 % des non-recommandations détectées.
- **L'arbre de décision a un profil atypique.** Son rappel et sa spécificité sont voisins (environ 0,93 chacun), d'où l'accuracy équilibrée la plus élevée du tableau, et sa précision sur la classe 1 est très haute (0,98). Il est en revanche le moins fiable lorsqu'il annonce « non recommandé » : cette annonce n'est correcte que dans 74 % des cas, contre 83 % pour le Linear SVC.
- **Aucun modèle n'est meilleur partout.** Le choix du « meilleur » modèle dépend du coût relatif des deux types d'erreur : manquer un avis non recommandé (FP) ou en signaler un à tort (FN).

Le Tableau 8.5 montre que le classement des modèles dépend de la métrique : le Linear SVC est premier selon le F1, le macro-F1 et l'AUC (où il devance de peu l'arbre de décision en macro-F1, 0,893 contre 0,891, soit un écart inférieur à l'incertitude de la reconstitution), mais c'est l'arbre de décision qui l'emporte en accuracy équilibrée.

***Tableau 8.5** — Classement des cinq modèles appris selon la métrique retenue (valeurs reconstituées pour le macro-F1 et l'accuracy équilibrée)*

Rang	F1 (classe 1)	Macro-F1	Accuracy équilibrée	ROC-AUC
1	Linear SVC	Linear SVC	Decision Tree	Linear SVC
2	Random Forest	Decision Tree	Linear SVC	Random Forest
3	Decision Tree	Random Forest	Random Forest	Decision Tree
4	K-NN	K-NN	Multinomial NB	K-NN
5	Multinomial NB	Multinomial NB	K-NN	Multinomial NB

8.5 Courbes ROC et AUC

L'AUC rend compte du *classement* produit par chaque modèle, avant toute décision de seuil. Les cinq modèles appris atteignent des AUC comprises entre 0,9531 (Naïve Bayes) et 0,9775 (Linear SVC), c'est-à-dire bien au-dessus de la valeur 0,5 d'un classement aléatoire. L'AUC du Linear SVC se calcule à partir de sa fonction de décision, celle de l'arbre à partir des proportions de classes dans les feuilles (une courbe en escalier, car un arbre ne produit qu'un nombre limité de scores distincts), celle de la forêt à partir de la moyenne des probabilités des arbres. L'AUC étant indépendante du seuil, elle suggère qu'un **réglage du seuil de décision** pourrait améliorer le rappel de la classe 0 sans changer de modèle, au prix d'une baisse du rappel de la classe 1 : c'est l'une des perspectives du projet.

8.6 Positionnement par rapport à des références externes et rôle de la note

Les résultats gagnent à être replacés dans le contexte des travaux publiés sur le même jeu de données. Une étude indépendante (McCormick, 2021) rapporte, sur un jeu de test de 2 350 avis issu d'un autre découpage, les F1-scores du Tableau 8.6. Ces valeurs ne sont qu'un **ordre de grandeur** : le jeu de test diffère de celui du projet (sa proportion de classe 1 est d'environ 0,83, contre 0,8223) et l'incertitude d'un F1 mesuré sur 2 350 avis est de l'ordre de 0,005. Les approches de Deep Learning y figurent uniquement comme repères de comparaison : le projet n'en utilise aucune.

***Tableau 8.6** — F1-scores de références externes et du projet (jeux de test différents : comparaison indicative)*

Stratégie	F1-score (classe 1)	Source
Toujours prédire « recommandé »	0,906	Étude externe (test de 2 350 avis)
Zero-R du projet	0,9025	Projet (test d'environ 4 700 avis)
Règle : recommandé si Rating ≥ 3	0,953	Étude externe
Modèle de langage sur le seul texte	0,945	Étude externe (référence, hors périmètre du projet)
Arbres boostés sur les variables hors texte	0,965	Étude externe
Linear SVC du projet (texte + métadonnées)	**0,9622**	**Projet**
Modèle de langage sur texte + métadonnées	0,968	Étude externe (référence, hors périmètre du projet)

Trois conclusions s'en dégagent.

1. **Le résultat du projet est cohérent avec la littérature.** Un F1 de 0,9622 se situe dans la fourchette publiée (0,945 à 0,968) pour des approches bien plus lourdes : rien n'indique ni sous-performance ni fuite de données.
2. **La note concentre l'essentiel du signal.** Une règle triviale (« recommandé si la note est au moins 3 ») atteint déjà un F1 de l'ordre de 0,95, et un modèle d'arbres sans aucune variable textuelle atteint 0,965. L'étude citée observe d'ailleurs que presque tous les avis notés de 3 à 5 sont recommandés et presque tous ceux notés 1 ou 2 ne le sont pas. C'est cohérent avec le test de Mann-Whitney du chapitre chapitre 5, et cela relativise le gain apporté par le texte sur le seul F1 de la classe 1.
3. **L'intérêt du texte est ailleurs.** Il réside dans les cas où la note ne suffit pas à trancher, dans l'explicabilité (les termes les plus discriminants) et surtout dans les situations où la note est absente : le modèle de langage sur le seul texte gagne alors près de 0,04 de F1 sur le baseline. Mesurer cet apport sur le pipeline du projet demanderait une **ablation** (entraînement sans la variable Rating), que nous proposons en perspective.

8.7 Bilan du chapitre

- Le Linear SVC est le meilleur modèle (accuracy 0,9378 ; F1 0,9622 ; AUC 0,9775) et réduit de 65 % le taux d’erreur du baseline.
- L’écart avec la forêt aléatoire est inférieur à l’incertitude d’échantillonnage ; le choix du Linear SVC repose aussi sur des critères pratiques.
- L’analyse par classe, reconstituée, montre que la classe minoritaire reste la plus difficile : environ 18 % des avis non recommandés échappent au modèle retenu.
- La note porte l’essentiel du signal ; mesurer l’apport propre du texte est la prochaine étape naturelle.

9 Application Streamlit

Le dernier maillon du pipeline est une application web interactive, `src/app.py`, qui permet à une personne non spécialiste de tester le modèle en direct et de consulter les résultats de l'étude. Ce chapitre décrit son architecture, ses six onglets, la chaîne de prédiction et les évolutions envisagées.

9.1 Architecture et principes de fonctionnement

Streamlit est une bibliothèque Python qui construit une interface web à partir d'un simple script, sans écrire de HTML ni de JavaScript. Son modèle d'exécution est particulier : *à chaque interaction de l'utilisateur (clic, saisie, changement d'onglet), le script est ré-exécuté de haut en bas*. Recharger un modèle depuis le disque à chaque interaction serait donc très coûteux. L'application évite ce piège en chargeant les fichiers `models/best_model.pkl` et `models/preprocessor.pkl` dans une fonction décorée par `@st.cache_resource` : l'objet est conservé en mémoire d'une exécution à l'autre, et le chargement n'a lieu qu'une seule fois (Figure 9.1).

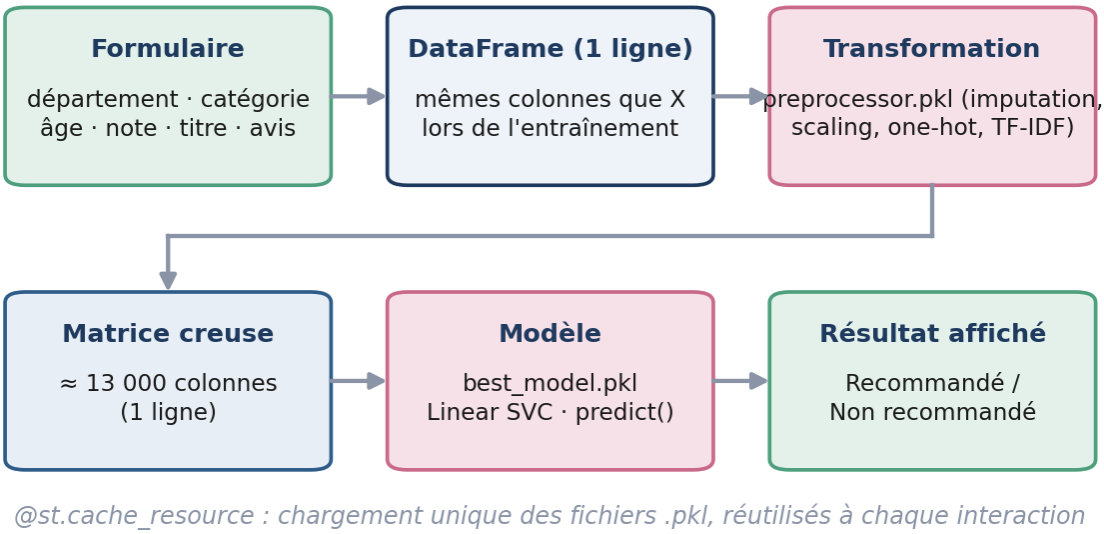


***Figure 9.1** — Chaîne de traitement de l'application : du formulaire à la prédiction affichée.*

- **Chemins robustes.** L'application résout le dossier `models/` à partir de l'emplacement du script, avec `Path(__file__).resolve().parents[1]` : `parents[0]` désigne le dossier `src/` et `parents[1]` la racine du projet. Elle fonctionne ainsi quel que soit le répertoire depuis lequel elle est lancée, sans configuration manuelle.
- **Cohérence modèle-préprocesseur.** Le modèle et le préprocesseur sont chargés ensemble : le modèle n'a de sens que sur des données transformées par le préprocesseur avec lequel il a été entraîné (section section 7.4).
- **Thème visuel.** Les couleurs (rose poudré, fond clair) sont définies dans `src/config.toml`.

9.2 Les six onglets

L'interface est organisée en six onglets, décrits dans le Tableau 9.1.

Tableau 9.1 — Les six onglets de l'application

N°	Onglet	Contenu
1	Accueil	Présentation du projet, de son objectif et de la démarche
2	Prédiction en direct	Saisie d'un profil (département, catégorie, âge, note, avis) et prédiction en temps réel par `model.predict()` sur les données transformées par `preprocessor.pkl`
3	Performance des modèles	Tableau comparatif des modèles et graphique du F1-score
4	Statistiques du dataset	Répartition de la variable cible et répartition par département
5	Découvertes clés	Synthèse des enseignements : déséquilibre des classes, modèle champion, limites de l'arbre de décision, apport du réglage des hyperparamètres
6	À propos	Stack technique et avertissement académique

9.3 Chaîne de prédiction en direct

L'onglet « Prédiction en direct » est le cœur fonctionnel de l'application. Il suit toujours la même séquence :

1. l'utilisatrice renseigne le formulaire : département, catégorie de produit, âge, note et texte de l'avis ;
2. les valeurs sont assemblées en un **DataFrame d'une ligne** dont les colonnes reprennent le schéma des variables explicatives de l'entraînement ;
3. le préprocesseur applique exactement les mêmes transformations que lors de l'apprentissage (imputation, standardisation, encodage one-hot, TF-IDF avec le vocabulaire et les idf de l'entraînement) et produit une matrice creuse d'une ligne ;
4. le Linear SVC calcule sa fonction de décision et renvoie la classe prédite via `model.predict()` ;
5. l'interface affiche le résultat (« recommandé » ou « non recommandé »).

Deux points techniques méritent d'être soulignés. D'abord, le préprocesseur attend toutes les colonnes du schéma d'entraînement : les variables que le formulaire ne demande pas (par exemple le titre ou le nombre de retours positifs) doivent être complétées par des valeurs neutres avant la transformation, ce que les imputeurs et le traitement du texte vide rendent possible. Ensuite, le Linear SVC n'expose pas `predict_proba` : il fournit un score de décision, dont le signe donne la classe. L'affichage d'un score de confiance suppose donc d'exploiter ce score (distance à l'hyperplan) ou de calibrer le modèle (section section 10.3).

> **Cohérence entraînement-service**
>
> Une erreur classique en production est le *training-serving skew* : appliquer à l'application des transformations légèrement différentes de celles de l'entraînement. En sérialisant le préprocesseur ajusté et en le rechargeant tel quel, le projet élimine ce risque.

9.4 Exécution et déploiement

Après installation des dépendances, l'application se lance depuis la racine du projet avec la commande suivante ; Streamlit ouvre alors l'interface dans le navigateur.

```
pip install -r requirements.txt
streamlit run src/app.py
```

L'exécution locale est la modalité retenue, conformément au cahier des charges, pour lequel le déploiement en ligne est optionnel. Les fichiers `.pkl` étant versionnés dans `models/`, l'application démarre immédiatement après le clonage, sans avoir à réexécuter les notebooks.

9.5 Évolutions envisagées

L'application remplit son rôle de démonstration. Plusieurs évolutions la rapprocheraient d'un véritable outil d'aide à la décision et des exigences complètes du cahier des charges :

- **Dashboard d'exploration complet.** Le cahier des charges prévoit six éléments minimaux : tableau de statistiques descriptives, informations sur les valeurs manquantes, distributions des variables numériques (histogrammes et KDE), fréquences des variables catégorielles, matrice de corrélation et au moins un filtre interactif. L'onglet « Statistiques du dataset » couvre aujourd'hui la répartition de la cible et par département ; le compléter est la suite logique.
- **Score de confiance.** Afficher la distance à l'hyperplan, ou une probabilité calibrée (`CalibratedClassifierCV`), pour que l'utilisatrice sache si une prédiction est tranchée ou proche de la frontière.
- **Explicabilité.** Mettre en évidence les termes qui ont le plus pesé dans la décision, grâce aux coefficients du Linear SVC. Un LLM pourrait aussi rédiger une explication en langage naturel, ce que le cahier des charges autorise comme outil applicatif, à condition que le moteur de prédiction reste un modèle de Machine Learning classique.
- **Captures d'écran et déploiement.** Documenter l'interface dans le rapport et, si souhaité, la publier en ligne.

10 Discussion, limites et perspectives

Ce chapitre prend du recul : il confronte les résultats aux questions posées en introduction, recense les limites du travail avec les mesures correctives correspondantes, puis propose des perspectives classées par intérêt.

10.1 Retour sur les questions du projet

Tableau 10.1 — Réponses aux cinq questions du projet

Question posée	Réponse apportée	Chapitre
1. Quelles variables sont liées à la recommandation ?	La note (test de Mann-Whitney U) et le département (test du χ^2) sont étudiés ; la note concentre l'essentiel du signal, comme le confirment les références externes.	chapitre 5, chapitre 8
2. Comment combiner des variables hétérogènes sans fuite de données ?	ColumnTransformer en trois branches, ajusté sur l'entraînement seul. Une fuite mineure subsiste dans la validation croisée (préprocesseur ajusté avant celle-ci), sans effet sur le jeu de test.	chapitre 6
3. Quel algorithme offre le meilleur compromis ?	Le Linear SVC (accuracy 0,9378 ; F1 0,9622 ; AUC 0,9775), indiscernable de la forêt aléatoire à l'incertitude près mais plus léger et mieux adapté aux matrices creuses.	chapitre 7, chapitre 8
4. Comment évaluer honnêtement avec 82,2 % de classe majoritaire ?	Baseline Zero-R, F1 et AUC, analyse par classe : la classe minoritaire reste la plus difficile (rappel d'environ 0,82).	chapitre 8
5. Comment rendre le modèle utilisable ?	Application Streamlit à six onglets ; modèle et préprocesseur sérialisés et rechargés tels quels.	chapitre 9

10.2 Limites du travail

Tableau 10.2 — Limites identifiées et mesures correctives

Limite	Explication	Mesure envisagée
Langue unique	Les avis sont en anglais ; un texte dans une autre langue produirait des vecteurs presque vides (mots hors vocabulaire).	Prédictions fiables en anglais uniquement ; extension multilingue
Déséquilibre des classes	82,2 % / 17,8 % ; le modèle retenu détecte environ 82 % des non-recommandations.	Pondération des classes, ré-échantillonnage, réglage du seuil
Poids dominant de la note	Une règle fondée sur la seule note atteint un F1 de l'ordre de 0,95 dans les références externes.	Ablation sans Rating pour mesurer l'apport propre du texte
Sac de mots	Le TF-IDF ignore l'ordre des mots, la négation et l'ironie.	n-grammes ; réglage de la vectorisation
Un seul découpage train/test	Erreur-type d'environ 0,35 point sur l'accuracy : les écarts inférieurs ne sont pas concluants.	Validation croisée répétée, intervalles de confiance par bootstrap, test de McNemar
Préprocesseur ajusté avant la validation croisée	Les plis de validation influencent légèrement les idf et les moyennes.	Intégrer le préprocesseur dans un Pipeline passé à GridSearchCV
Pas de probabilités calibrées	LinearSVC ne fournit qu'un score de décision.	Calibration ; score de confiance dans l'application
SVM limité au noyau linéaire	Les noyaux RBF et polynomial ne figurent pas dans la comparaison.	Comparer SVC à noyaux non linéaires, avec un budget de calcul adapté
Représentativité	Un seul détaillant, une seule langue, une seule période.	Valider sur d'autres sources avant tout usage réel

Limite	Explication	Mesure envisagée
Équité	L'âge est une variable explicative du modèle.	Mesurer les performances par tranche d'âge

10.3 Perspectives

Les perspectives sont classées par rapport entre effort et bénéfice attendu. Elles respectent le cadre du cahier des charges : seuls des algorithmes de Machine Learning classique sont envisagés comme modèles prédictifs.

***Tableau 10.3** — Perspectives classées par priorité*

Priorité	Action	Bénéfice attendu
1	Régler le seuil de décision et tester la pondération des classes (`class_weight`)	Meilleur rappel de la classe « non recommandé », au prix d'une baisse maîtrisée de celui de la classe 1
2	Mener des ablations : sans la variable Rating, puis sans le texte	Quantifier l'apport propre du texte et celui des métadonnées
3	Exporter et analyser les matrices de confusion et les *classification reports* ; examiner des avis mal classés	Remplacer les valeurs reconstituées par les valeurs exactes et comprendre les erreurs
4	Placer le préprocesseur dans un `Pipeline` ; validation croisée répétée ; test de McNemar	Supprimer la fuite mineure et départager rigoureusement les modèles proches
5	Enrichir le TF-IDF : n-grammes (1 à 2), `min_df`, pondération sous-linéaire	Capter la négation et les expressions figées (« not good », « runs small »)
6	Tester d'autres algorithmes classiques : régression logistique, gradient boosting, SVC à noyaux non linéaires	Probabilités natives (régression logistique) ; couverture complète du périmètre du cours
7	Afficher un score de confiance calibré et les termes les plus influents dans l'application	Confiance et explicabilité des prédictions
8	Compléter le dashboard d'exploration (six éléments du cahier des charges)	Exploration interactive complète des données
9	Étendre à d'autres langues ; déployer l'application en ligne	Généralisation et accessibilité

10.4 Responsabilité et usage

Le modèle déduit la classe d'opinion la plus plausible à partir de signaux linguistiques et de métadonnées ; il ne juge ni une cliente ni un produit. Son usage doit rester celui d'un outil d'aide à l'analyse d'avis, avec une supervision humaine, et être précédé d'une validation sur les données réelles de l'organisation qui l'emploierait. Comme tout modèle entraîné sur des textes d'un seul public, il peut refléter les biais de cette population (section section 3.6).

11 Conclusion

Ce projet a conduit un pipeline de Data Mining de bout en bout, des 23 486 avis bruts du jeu *Women's E-Commerce Clothing Reviews* jusqu'à une application Streamlit capable de prédire en direct si une cliente recommande un article. Il répond au cahier des charges du parcours A : un jeu de données de volumétrie suffisante, une exploration documentée, deux tests statistiques justifiés, un prétraitement sans fuite de données, la comparaison de six approches de Machine Learning classique avec optimisation systématique des hyperparamètres, et un dépôt GitHub structuré.

Sur le plan des résultats, le **Linear SVC** s'impose avec une accuracy de 0,9378, un F1-score de 0,9622 et une ROC-AUC de 0,9775, soit une réduction de 65 % du taux d'erreur par rapport au baseline Zero-R. La reconstitution des métriques par classe apporte un éclairage complémentaire : le modèle détecte environ 82 % des avis non recommandés, la classe minoritaire restant la plus difficile à repérer. La mise en perspective avec des travaux externes montre que le résultat est cohérent avec l'état de l'art sur ce jeu de données, et que la note concentre l'essentiel du signal.

Sur le plan méthodologique, le projet illustre quatre enseignements : **(1)** un baseline trivial est indispensable pour apprécier la valeur ajoutée d'un modèle, surtout sur des classes déséquilibrées ; **(2)** l'accuracy seule ne suffit pas, et le choix du « meilleur » modèle dépend de la métrique et du coût des erreurs ; **(3)** un écart de performance doit toujours être comparé à l'incertitude d'échantillonnage ; **(4)** la reproductibilité (versions figées, graines aléatoires, découpage stratifié, artefacts sérialisés) est une exigence scientifique autant que technique.

Les prolongements les plus prometteurs sont le réglage du seuil de décision et la pondération des classes pour mieux détecter les non-recommandations, les ablations pour mesurer l'apport propre du texte, l'enrichissement du TF-IDF par des n-grammes, et la complétion de l'application (score de confiance, explicabilité, dashboard d'exploration complet). Ces pistes s'inscrivent toutes dans le cadre du Machine Learning classique retenu pour ce mini-projet.

Références bibliographiques

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Breiman, L., Friedman, J., Olshen, R. et Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.

Brooks, N. (2018). *Women's E-Commerce Clothing Reviews* [jeu de données]. Kaggle. https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. et Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

Chawla, N. V., Bowyer, K. W., Hall, L. O. et Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

Cortes, C. et Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

Cover, T. et Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.

Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R. et Lin, C.-J. (2008). LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research*, 9, 1871–1874.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.

Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. *Proceedings of ECML-98*, Lecture Notes in Computer Science 1398, 137–142. Springer.

Mann, H. B. et Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1), 50–60.

McCallum, A. et Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. *AAAI-98 Workshop on Learning for Text Categorization*, 41–48.

McCormick, C. (2021). *Combining categorical and numerical features with text in BERT*. https://mccormickml.com/2021/06/29/combining-categorical-numerical-features-with-bert

OpenML (s. d.). *Women's E-Commerce Clothing Reviews*, jeu de données n° 43673 (valeurs distinctes et manquantes). https://api.openml.org/d/43673

Pearson, K. (1900). On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *Philosophical Magazine*, 50(302), 157–175.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Salton, G. et Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.

Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

Documentation de scikit-learn, https://scikit-learn.org ; documentation de Streamlit, https://docs.streamlit.io ; documentation de SciPy, https://docs.scipy.org (consultées pour les paramètres et le comportement des implémentations).

A Procédure de reproduction pas à pas

Cette annexe détaille la procédure à suivre pour régénérer l'ensemble du projet à partir du dépôt. Prérequis : Python 3, Git et un accès au fichier `reviews.csv` (jeu Kaggle).

A.1 Installation de l'environnement

```
git clone https://github.com/RSD018/Prediction-des-recommandations--dachats--de-vetements--pour--femmes.git
```

```
cd Prediction-des-recommandations--dachats--de-vetements--pour--femmes
python -m venv venv
source venv/bin/activate        # Windows : venv\Scripts\activate
pip install -r requirements.txt
```

Le dossier créé par `git clone` porte le nom du dépôt ; c'est dans ce dossier que se placent toutes les commandes suivantes. L'installation depuis `requirements.txt` fige les versions exactes des bibliothèques, ce qui garantit la compatibilité des fichiers `.pkl`.

A.2 Régénération des données et des modèles

1. Placer `reviews.csv` dans `data/raw/`.
2. Lancer JupyterLab (`jupyter lab`) et exécuter, dans l'ordre et en entier, `01_eda_and_cleaning.ipynb`, puis `02_preprocessing_encoding.ipynb`, puis `03_classification_models.ipynb`.
3. Vérifier la présence des fichiers produits (Tableau A.1).
4. Lancer l'application : `streamlit run src/app.py`.

__Tableau A.1__ — Fichiers produits par chaque notebook et contrôles de cohérence

Après le notebook	Fichier attendu	Contrôle rapide
01	`data/processed/reviews_clean.csv`	Le fichier existe ; il compte pratiquement autant de lignes que le jeu brut
02	`models/preprocessor.pkl` (+ vectoriseurs TF-IDF)	Les fichiers existent ; la matrice d'entraînement compte environ 13 000 colonnes
03	`models/best_model.pkl`	Le tableau de résultats retrouve le Linear SVC en tête (accuracy 0,9378)

> **Si les résultats diffèrent légèrement**
>
> Des écarts de dernière décimale peuvent apparaître si les versions de bibliothèques diffèrent de celles de `requirements.txt`, ou si un modèle à composante aléatoire est exécuté sans la graine `random_state=42`. Réinstaller l'environnement depuis `requirements.txt` est la première vérification à faire.

B Dérivation des métriques reconstituées

Cette annexe démontre les formules de la section section 8.4. Toutes les grandeurs VP, FP, FN et VN sont exprimées en fraction du jeu de test (leur somme vaut 1). On note A l'accuracy, F_1 le F1-score de la classe 1 et p la proportion de la classe 1 dans le jeu de test.

B.1 Hypothèses et relations de départ

- La classe positive est la classe 1, de proportion p = VP + FN ; la classe 0 a pour proportion 1 − p = VN + FP.
- L'accuracy est la part des prédictions correctes : A = VP + VN, donc **le taux d'erreur vaut FP + FN = 1 − A**.
- Le F1-score de la classe 1 vaut F_1 = 2VP / (2VP + FP + FN).

B.2 Démonstration

Étape 1. Réécrivons la définition du F1 pour isoler FP + FN :

$$F_1 = \frac{2\,VP}{2\,VP + FP + FN} \;\Rightarrow\; FP + FN = \frac{2\,VP\,(1 - F_1)}{F_1} \tag{B.1}$$

Étape 2. Or FP + FN = 1 − A. En égalant les deux expressions, on isole VP :

$$1 - A = \frac{2\,VP\,(1 - F_1)}{F_1} \;\Rightarrow\; VP = \frac{F_1\,(1 - A)}{2\,(1 - F_1)} \tag{B.2}$$

Étape 3. Les autres effectifs s'en déduisent par les relations de départ :

$$FN = p - VP \qquad FP = (1 - A) - FN = VP - (A - 1 + p) \qquad VN = 1 - p - FP \tag{B.3}$$

Vérification sur le baseline Zero-R. Il prédit toujours la classe 1, donc A = p et F_1 = 2p/(1 + p), d'où 1 − F_1 = (1 − p)/(1 + p). La formule donne VP = [2p/(1 + p)] (1 − p) / [2(1 − p)/(1 + p)] = p, puis FN = 0 et FP = 1 − p : c'est bien le comportement attendu (aucun avis non recommandé détecté).

B.3 Application numérique au Linear SVC

__Tableau B.1__ — Reconstitution détaillée de la matrice de confusion du Linear SVC

Grandeur	Calcul	Valeur
VP	0,9622 × (1 − 0,9378) / [2 × (1 − 0,9622)]	0,7917
FN	p − VP = 0,8223 − 0,7917	0,0306
FP	VP − (A − 1 + p) = 0,7917 − (0,9378 − 1 + 0,8223)	0,0316
VN	1 − p − FP = 1 − 0,8223 − 0,0316	0,1461
Précision (cl. 1)	VP / (VP + FP)	0,9617
Rappel (cl. 1)	VP / p	0,9627
Spécificité (rappel cl. 0)	VN / (1 − p)	0,8224

B.4 Incertitude due à l’arrondi

Les métriques du README sont arrondies à 10^{-4}, donc connues à $\pm\ 5 \times 10^{-5}$. La propagation de cette incertitude à VP passe par les dérivées partielles :

$$\frac{\partial VP}{\partial F_1} = \frac{1 - A}{2(1 - F_1)^2} \qquad \frac{\partial VP}{\partial A} = -\frac{F_1}{2(1 - F_1)} \tag{B.4}$$

Pour le Linear SVC, ces dérivées valent 21,8 et −12,7 : l’erreur maximale sur VP est donc de $(21{,}8 + 12{,}7) \times 5 \times 10^{-5} \approx 1{,}7 \times 10^{-3}$ (soit 0,17 point de pourcentage). L’erreur sur FP est du même ordre. Rapportée à la proportion de la classe 0 ($1 - p = 0{,}1777$), elle conduit à une incertitude d’environ 0,010 sur la spécificité, tandis que le rappel et la précision, calculés sur des effectifs plus grands, sont connus à moins de 0,003. C’est ce qui justifie l’incertitude de ± 0,01 annoncée pour l’ensemble des valeurs reconstituées, qui est une borne supérieure.

C Glossaire

Tableau C.1 — Glossaire des termes techniques

Terme	Définition
Accuracy	Part des prédictions correctes sur l'ensemble des observations.
AUC (ROC-AUC)	Aire sous la courbe ROC ; probabilité qu'un exemple positif reçoive un score supérieur à celui d'un exemple négatif.
Baseline	Modèle de référence très simple (ici Zero-R) servant de plancher de comparaison.
Bag-of-words	Représentation d'un texte par les effectifs de ses mots, sans tenir compte de leur ordre.
ColumnTransformer	Objet scikit-learn appliquant des transformations différentes à des groupes de colonnes puis assemblant les résultats.
CRISP-DM	Méthode de référence en six phases pour conduire un projet de fouille de données.
EDA	Analyse exploratoire des données : description, visualisation et diagnostic de qualité avant modélisation.
F1-score	Moyenne harmonique de la précision et du rappel.
Fuite de données	Influence, directe ou indirecte, du jeu de test sur l'apprentissage ou le prétraitement.
GridSearchCV	Recherche exhaustive des hyperparamètres par validation croisée sur une grille de valeurs.
Hyperparamètre	Réglage fixé avant l'apprentissage (K, profondeur, C, α…) qui pilote la complexité du modèle.
IQR	Écart interquartile Q3 − Q1 ; base de la règle de Tukey pour les valeurs atypiques.
joblib	Bibliothèque Python utilisée pour sérialiser et recharger les modèles et les préprocesseurs.
KDE	Estimation de densité par noyau ; version lissée de l'histogramme.
Matrice creuse	Matrice dont la plupart des valeurs sont nulles ; seules les valeurs non nulles sont stockées.
One-hot	Encodage d'une variable catégorielle en colonnes binaires, une par modalité.
p-valeur	Probabilité d'observer un résultat au moins aussi extrême si l'hypothèse nulle était vraie.
Précision	Parmi les exemples prédits positifs, part de ceux qui le sont réellement.
Rappel (sensibilité)	Parmi les exemples réellement positifs, part de ceux qui sont détectés.
Spécificité	Parmi les exemples réellement négatifs, part de ceux qui sont détectés ; rappel de la classe 0.
Stratification	Découpage conservant, dans chaque sous-ensemble, les proportions de classes du jeu complet.
Surapprentissage	Le modèle mémorise le bruit de l'entraînement et généralise mal à de nouvelles données.
TF-IDF	Pondération d'un terme par sa fréquence dans le document, atténuée par sa fréquence dans le corpus.
Validation croisée	Évaluation répétée où chaque sous-ensemble sert tour à tour de validation, les autres servant à l'apprentissage.
Z-score	Valeur centrée et réduite : (x − moyenne) / écart-type.